

# **Introduction à R et au tidyverse**

Julien Barnier

2024-12-19



# Table des matières

|                                                                        |               |
|------------------------------------------------------------------------|---------------|
| <b>À propos de ce document</b>                                         | <b>1</b>      |
| Remerciements . . . . .                                                | 1             |
| Licence . . . . .                                                      | 1             |
| <br><b>I. Introduction</b>                                             | <br><b>3</b>  |
| <b>1. Présentation</b>                                                 | <b>5</b>      |
| 1.1. À propos de R . . . . .                                           | 5             |
| 1.2. À propos de RStudio . . . . .                                     | 6             |
| 1.3. À propos du <i>tidyverse</i> . . . . .                            | 6             |
| 1.4. Structure du document . . . . .                                   | 6             |
| 1.5. Prérequis . . . . .                                               | 6             |
| <br><b>II. Introduction à R</b>                                        | <br><b>9</b>  |
| <b>2. Prise en main</b>                                                | <b>11</b>     |
| 2.1. La console . . . . .                                              | 11            |
| 2.1.1. L'invite de commandes . . . . .                                 | 11            |
| 2.1.2. Précisions concernant la saisie des commandes . . . . .         | 13            |
| 2.2. Objets . . . . .                                                  | 13            |
| 2.2.1. Objets simples . . . . .                                        | 13            |
| 2.2.2. Vecteurs . . . . .                                              | 15            |
| 2.3. Fonctions . . . . .                                               | 18            |
| 2.3.1. Principe . . . . .                                              | 18            |
| 2.3.2. Arguments . . . . .                                             | 19            |
| 2.3.3. Aide sur une fonction . . . . .                                 | 20            |
| 2.4. Regrouper ses commandes dans des scripts . . . . .                | 21            |
| 2.4.1. Commentaires . . . . .                                          | 22            |
| 2.5. Installer et charger des extensions ( <i>packages</i> ) . . . . . | 22            |
| 2.6. Exercices . . . . .                                               | 24            |
| <br><b>3. Premier travail avec des données</b>                         | <br><b>27</b> |
| 3.1. Jeu de données d'exemple . . . . .                                | 27            |
| 3.2. Tableau de données ( <i>data frame</i> ) . . . . .                | 28            |
| 3.2.1. Structure du tableau . . . . .                                  | 30            |
| 3.2.2. Accéder aux variables d'un tableau . . . . .                    | 32            |
| 3.2.3. Créer une nouvelle variable . . . . .                           | 33            |
| 3.3. Analyse univariée . . . . .                                       | 33            |
| 3.3.1. Analyser une variable quantitative . . . . .                    | 33            |
| 3.3.2. Analyser une variable qualitative . . . . .                     | 38            |

|                                                                                     |           |
|-------------------------------------------------------------------------------------|-----------|
| 3.4. Exercices . . . . .                                                            | 43        |
| <b>4. Analyse bivariable</b>                                                        | <b>45</b> |
| 4.1. Croisement de deux variables qualitatives . . . . .                            | 45        |
| 4.1.1. Tableaux croisés . . . . .                                                   | 45        |
| 4.1.2. Test du $\chi^2$ . . . . .                                                   | 47        |
| 4.1.3. Représentation graphique . . . . .                                           | 48        |
| 4.2. Croisement d'une variable quantitative et d'une variable qualitative . . . . . | 50        |
| 4.2.1. Représentation graphique . . . . .                                           | 50        |
| 4.2.2. Calculs d'indicateurs . . . . .                                              | 52        |
| 4.2.3. Tests statistiques . . . . .                                                 | 53        |
| 4.3. Croisement de deux variables quantitatives . . . . .                           | 55        |
| 4.3.1. Représentation graphique . . . . .                                           | 56        |
| 4.3.2. Calcul d'indicateurs . . . . .                                               | 58        |
| 4.3.3. Régression linéaire . . . . .                                                | 61        |
| 4.4. Exercices . . . . .                                                            | 63        |
| <b>5. Organiser ses scripts</b>                                                     | <b>65</b> |
| 5.1. Les projets dans RStudio . . . . .                                             | 65        |
| 5.2. Créer des sections dans un script . . . . .                                    | 67        |
| 5.3. Répartir son travail entre plusieurs scripts . . . . .                         | 68        |
| 5.4. Désactiver la sauvegarde de l'espace de travail . . . . .                      | 69        |
| <b>III. Introduction au tidyverse</b>                                               | <b>73</b> |
| <b>6. Le tidyverse</b>                                                              | <b>75</b> |
| 6.1. Extensions . . . . .                                                           | 75        |
| 6.2. Installation . . . . .                                                         | 75        |
| 6.3. tidy data . . . . .                                                            | 76        |
| 6.4. tibbles . . . . .                                                              | 77        |
| <b>7. Importer et exporter des données</b>                                          | <b>81</b> |
| 7.1. Import de fichiers textes . . . . .                                            | 81        |
| 7.1.1. Interface interactive d'import de fichiers . . . . .                         | 82        |
| 7.2. Import depuis un fichier Excel . . . . .                                       | 86        |
| 7.3. Import de fichiers SAS, SPSS et Stata . . . . .                                | 87        |
| 7.4. Import de fichiers dBase . . . . .                                             | 88        |
| 7.5. Connexion à des bases de données . . . . .                                     | 88        |
| 7.5.1. Interfaçage via l'extension DBI . . . . .                                    | 88        |
| 7.5.2. Utilisation de <code>dplyr</code> et <code>dbplyr</code> . . . . .           | 90        |
| 7.5.3. Ressources . . . . .                                                         | 92        |
| 7.6. Export de données . . . . .                                                    | 92        |
| 7.6.1. Export de tableaux de données . . . . .                                      | 92        |
| 7.6.2. Sauvegarder des objets . . . . .                                             | 92        |
| <b>8. Visualiser avec ggplot2</b>                                                   | <b>95</b> |
| 8.1. Préparation . . . . .                                                          | 95        |
| 8.2. Initialisation . . . . .                                                       | 96        |

|                                                                     |            |
|---------------------------------------------------------------------|------------|
| 8.3. Exemples de <code>geom</code> . . . . .                        | 99         |
| 8.3.1. <code>geom_boxplot</code> . . . . .                          | 99         |
| 8.3.2. <code>geom_violin</code> . . . . .                           | 101        |
| 8.3.3. <code>geom_bar</code> et <code>geom_col</code> . . . . .     | 102        |
| 8.3.4. <code>geom_text</code> . . . . .                             | 106        |
| 8.3.5. <code>geom_label</code> . . . . .                            | 107        |
| 8.3.6. <code>geom_density</code> . . . . .                          | 108        |
| 8.3.7. <code>geom_line</code> . . . . .                             | 109        |
| 8.3.8. <code>geom_hex</code> et <code>geom_bin2d</code> . . . . .   | 110        |
| 8.4. Mappages . . . . .                                             | 114        |
| 8.4.1. Exemples de mappages . . . . .                               | 114        |
| 8.4.2. <code>aes()</code> or not <code>aes()</code> ? . . . . .     | 116        |
| 8.4.3. <code>geom_bar</code> et <code>position</code> . . . . .     | 118        |
| 8.5. Représentation de plusieurs <code>geom</code> . . . . .        | 121        |
| 8.5.1. Plusieurs sources de données . . . . .                       | 125        |
| 8.6. Faceting . . . . .                                             | 127        |
| 8.7. Scales . . . . .                                               | 130        |
| 8.7.1. <code>scale_size</code> . . . . .                            | 131        |
| 8.7.2. <code>scale_x</code> , <code>scale_y</code> . . . . .        | 133        |
| 8.7.3. <code>scale_color</code> , <code>scale_fill</code> . . . . . | 137        |
| 8.8. Thèmes . . . . .                                               | 145        |
| 8.9. L'add-in <code>esquisse</code> . . . . .                       | 148        |
| 8.10. Ressources . . . . .                                          | 152        |
| 8.11. Exercices . . . . .                                           | 153        |
| <b>9. Recoder des variables</b> . . . . .                           | <b>161</b> |
| 9.1. Rappel sur les variables et les vecteurs . . . . .             | 161        |
| 9.2. Tests et comparaison . . . . .                                 | 163        |
| 9.3. Recoder une variable qualitative . . . . .                     | 165        |
| 9.3.1. Facteurs et <code>forcats</code> . . . . .                   | 166        |
| 9.3.2. Modifier les modalités d'une variable qualitative . . . . .  | 166        |
| 9.3.3. Ordonner les modalités d'une variable qualitative . . . . .  | 171        |
| 9.4. Combiner plusieurs variables . . . . .                         | 174        |
| 9.4.1. <code>ifelse</code> . . . . .                                | 175        |
| 9.4.2. <code>case_when</code> . . . . .                             | 175        |
| 9.5. Découper une variable numérique en classes . . . . .           | 176        |
| 9.5.1. Interface graphique . . . . .                                | 177        |
| 9.6. Exercices . . . . .                                            | 179        |
| 9.6.1. Préparation . . . . .                                        | 179        |
| 9.6.2. Vecteurs et tests . . . . .                                  | 179        |
| 9.6.3. Recodages de variable qualitative . . . . .                  | 180        |
| 9.6.4. Combiner plusieurs variables . . . . .                       | 181        |
| 9.6.5. Découper une variable numérique . . . . .                    | 182        |
| <b>10. Manipuler les données avec <code>dplyr</code></b> . . . . .  | <b>183</b> |
| 10.1. Préparation . . . . .                                         | 183        |
| 10.2. Les verbes de <code>dplyr</code> . . . . .                    | 184        |
| 10.2.1. <code>slice</code> . . . . .                                | 184        |
| 10.2.2. <code>filter</code> . . . . .                               | 185        |
| 10.2.3. <code>select</code> et <code>rename</code> . . . . .        | 187        |

|                                                                                                      |            |
|------------------------------------------------------------------------------------------------------|------------|
| 10.2.4. <code>arrange</code> . . . . .                                                               | 190        |
| 10.2.5. <code>mutate</code> . . . . .                                                                | 192        |
| 10.3. Enchaîner les opérations avec le <i>pipe</i> . . . . .                                         | 193        |
| 10.4. Opérations groupées . . . . .                                                                  | 196        |
| 10.4.1. <code>group_by</code> . . . . .                                                              | 196        |
| 10.4.2. <code>summarise</code> et <code>count</code> . . . . .                                       | 201        |
| 10.4.3. Grouper selon plusieurs variables . . . . .                                                  | 203        |
| 10.4.4. Dégroupage . . . . .                                                                         | 205        |
| 10.5. Autres fonctions utiles . . . . .                                                              | 207        |
| 10.5.1. <code>slice_sample</code> . . . . .                                                          | 207        |
| 10.5.2. <code>lead</code> et <code>lag</code> . . . . .                                              | 208        |
| 10.5.3. <code>distinct</code> et <code>n_distinct</code> . . . . .                                   | 209        |
| 10.5.4. <code>relocate</code> . . . . .                                                              | 211        |
| 10.6. Tables multiples . . . . .                                                                     | 212        |
| 10.6.1. Concaténation : <code>bind_rows</code> et <code>bind_cols</code> . . . . .                   | 212        |
| 10.6.2. Jointures . . . . .                                                                          | 214        |
| 10.6.3. Types de jointures . . . . .                                                                 | 219        |
| 10.7. Ressources . . . . .                                                                           | 223        |
| 10.8. Exercices . . . . .                                                                            | 223        |
| 10.8.1. Les verbes de base de <code>dplyr</code> . . . . .                                           | 224        |
| 10.8.2. Enchaîner des opérations . . . . .                                                           | 228        |
| 10.8.3. <code>group_by</code> et <code>summarise</code> . . . . .                                    | 229        |
| 10.8.4. Jointures . . . . .                                                                          | 232        |
| 10.8.5. Bonus . . . . .                                                                              | 234        |
| <b>11. Manipuler du texte avec <code>stringr</code></b> . . . . .                                    | <b>235</b> |
| 11.1. Expressions régulières . . . . .                                                               | 235        |
| 11.2. Concaténer des chaînes . . . . .                                                               | 236        |
| 11.3. Convertir en majuscules / minuscules . . . . .                                                 | 237        |
| 11.4. Découper des chaînes . . . . .                                                                 | 238        |
| 11.5. Extraire des sous-chaînes par position . . . . .                                               | 239        |
| 11.6. Détecter des motifs . . . . .                                                                  | 239        |
| 11.7. Extraire des motifs . . . . .                                                                  | 240        |
| 11.8. Remplacer des motifs . . . . .                                                                 | 241        |
| 11.9. Modificateurs de motifs . . . . .                                                              | 242        |
| 11.10. Ressources . . . . .                                                                          | 242        |
| 11.11. Exercices . . . . .                                                                           | 243        |
| <b>12. Mettre en ordre avec <code>tidyr</code></b> . . . . .                                         | <b>245</b> |
| 12.1. Tidy data . . . . .                                                                            | 245        |
| 12.2. Trois règles pour des données bien rangées . . . . .                                           | 246        |
| 12.3. Les verbes de <code>tidyr</code> . . . . .                                                     | 248        |
| 12.3.1. <code>pivot_longer</code> : transformer des colonnes en lignes . . . . .                     | 248        |
| 12.3.2. <code>pivot_wider</code> : transformer des lignes en colonnes . . . . .                      | 250        |
| 12.3.3. <code>separate</code> : séparer une colonne en plusieurs colonnes . . . . .                  | 252        |
| 12.3.4. <code>separate_rows</code> : séparer une colonne en plusieurs lignes . . . . .               | 253        |
| 12.3.5. <code>unite</code> : regrouper plusieurs colonnes en une seule . . . . .                     | 254        |
| 12.3.6. <code>extract</code> : créer de nouvelles colonnes à partir d'une colonne de texte . . . . . | 255        |
| 12.3.7. <code>complete</code> : compléter des combinaisons de variables manquantes . . . . .         | 256        |
| 12.4. Ressources . . . . .                                                                           | 258        |

|                                                                                        |                |
|----------------------------------------------------------------------------------------|----------------|
| <b>13. Diffuser et publier avec rmarkdown</b>                                          | <b>259</b>     |
| 13.1. Créer un nouveau document . . . . .                                              | 264            |
| 13.2. Éléments d'un document R Markdown . . . . .                                      | 264            |
| 13.2.1. En-tête (préambule) . . . . .                                                  | 265            |
| 13.2.2. Texte du document . . . . .                                                    | 266            |
| 13.2.3. Blocs de code R . . . . .                                                      | 267            |
| 13.2.4. Compiler un document ( <i>Knit</i> ) . . . . .                                 | 268            |
| 13.3. Personnaliser le document généré . . . . .                                       | 269            |
| 13.4. Options des blocs de code R . . . . .                                            | 271            |
| 13.4.1. Nom du bloc . . . . .                                                          | 271            |
| 13.4.2. Options . . . . .                                                              | 272            |
| 13.4.3. Modifier les options . . . . .                                                 | 273            |
| 13.4.4. Options globales . . . . .                                                     | 273            |
| 13.4.5. Mise en cache des résultats . . . . .                                          | 274            |
| 13.5. Rendu des tableaux . . . . .                                                     | 274            |
| 13.5.1. Tableaux croisés . . . . .                                                     | 274            |
| 13.5.2. Tableaux de données et tris à plat . . . . .                                   | 276            |
| 13.6. Modèles de documents . . . . .                                                   | 277            |
| 13.6.1. Slides . . . . .                                                               | 277            |
| 13.6.2. Templates . . . . .                                                            | 279            |
| 13.7. Ressources . . . . .                                                             | 281            |
| <br><b>IV. Aller plus loin</b>                                                         | <br><b>283</b> |
| <b>14. Écrire ses propres fonctions</b>                                                | <b>285</b>     |
| 14.1. Introduction et exemples . . . . .                                               | 285            |
| 14.1.1. Structure d'une fonction . . . . .                                             | 285            |
| 14.1.2. Exemple de fonction . . . . .                                                  | 289            |
| 14.1.3. Effets de bord et affichage de messages . . . . .                              | 292            |
| 14.1.4. Utilité des fonctions . . . . .                                                | 294            |
| 14.2. Arguments et résultat d'une fonction . . . . .                                   | 296            |
| 14.2.1. Définition des arguments . . . . .                                             | 296            |
| 14.2.2. Valeurs par défaut . . . . .                                                   | 297            |
| 14.2.3. Arguments obligatoires et arguments facultatifs . . . . .                      | 298            |
| 14.2.4. L'argument . . . . .                                                           | 298            |
| 14.2.5. Résultat d'une fonction . . . . .                                              | 300            |
| 14.3. Portée des variables . . . . .                                                   | 303            |
| 14.3.1. Une fonction peut accéder à un objet extérieur . . . . .                       | 303            |
| 14.3.2. Les arguments et les objets créés dans la fonction sont prioritaires . . . . . | 304            |
| 14.3.3. Un objet créé dans une fonction n'existe que dans cette fonction . . . . .     | 304            |
| 14.3.4. On ne peut pas modifier un objet global dans une fonction . . . . .            | 305            |
| 14.4. Les fonctions comme objets . . . . .                                             | 306            |
| 14.4.1. Passer des fonctions comme argument . . . . .                                  | 307            |
| 14.4.2. Fonctions anonymes . . . . .                                                   | 308            |
| 14.5. Ressources . . . . .                                                             | 309            |
| 14.6. Exercices . . . . .                                                              | 309            |
| 14.6.1. Introduction et exemples . . . . .                                             | 309            |
| 14.6.2. Arguments et résultat . . . . .                                                | 311            |
| 14.6.3. Portée des variables . . . . .                                                 | 312            |

|                                                                                                                  |            |
|------------------------------------------------------------------------------------------------------------------|------------|
| 14.6.4. Les fonctions comme objets . . . . .                                                                     | 313        |
| <b>15. dplyr avancé</b>                                                                                          | <b>315</b> |
| 15.1. Appliquer ses propres fonctions . . . . .                                                                  | 315        |
| 15.1.1. Exemple avec <code>mutate</code> . . . . .                                                               | 315        |
| 15.1.2. Exemple avec <code>summarise</code> . . . . .                                                            | 317        |
| 15.1.3. Exemple avec <code>rename_with</code> . . . . .                                                          | 320        |
| 15.2. <code>across()</code> : appliquer des fonctions à plusieurs colonnes . . . . .                             | 321        |
| 15.2.1. Appliquer une fonction à plusieurs colonnes . . . . .                                                    | 321        |
| 15.2.2. Passer des arguments supplémentaires à la fonction appliquée . . . . .                                   | 326        |
| 15.2.3. Noms des colonnes créées par un <code>mutate</code> . . . . .                                            | 327        |
| 15.2.4. Appliquer plusieurs fonctions à plusieurs colonnes . . . . .                                             | 328        |
| 15.2.5. Renommer plusieurs colonnes avec une fonction . . . . .                                                  | 329        |
| 15.3. Fonctions anonymes et syntaxes abrégées . . . . .                                                          | 330        |
| 15.4. <code>rowwise()</code> et <code>c_across()</code> : appliquer une transformation ligne par ligne . . . . . | 333        |
| 15.5. Ressources . . . . .                                                                                       | 338        |
| 15.6. Exercices . . . . .                                                                                        | 338        |
| 15.6.1. Appliquer ses propres fonctions . . . . .                                                                | 339        |
| 15.6.2. <code>across()</code> . . . . .                                                                          | 340        |
| 15.6.3. Fonctions anonymes et notations abrégées . . . . .                                                       | 341        |
| 15.6.4. <code>rowwise()</code> et <code>c_across()</code> . . . . .                                              | 343        |
| <b>16. Structures de données</b>                                                                                 | <b>345</b> |
| 16.1. Vecteurs atomiques . . . . .                                                                               | 345        |
| 16.1.1. Création d'un vecteur . . . . .                                                                          | 345        |
| 16.1.2. Vecteurs nommés . . . . .                                                                                | 346        |
| 16.1.3. Types de vecteurs . . . . .                                                                              | 347        |
| 16.1.4. Sélection d'éléments . . . . .                                                                           | 350        |
| 16.1.5. Modification . . . . .                                                                                   | 351        |
| 16.2. Listes . . . . .                                                                                           | 351        |
| 16.2.1. Création . . . . .                                                                                       | 352        |
| 16.2.2. Ajout d'éléments . . . . .                                                                               | 353        |
| 16.2.3. Sélection d'éléments . . . . .                                                                           | 354        |
| 16.2.4. Modification . . . . .                                                                                   | 356        |
| 16.2.5. Utilisation . . . . .                                                                                    | 358        |
| 16.3. Tableaux de données ( <i>data frame</i> et <i>tibble</i> ) . . . . .                                       | 358        |
| 16.3.1. Création . . . . .                                                                                       | 359        |
| 16.3.2. Noms de colonnes et de lignes . . . . .                                                                  | 359        |
| 16.3.3. Sélection de lignes et de colonnes . . . . .                                                             | 360        |
| 16.3.4. Modification . . . . .                                                                                   | 362        |
| 16.4. Ressources . . . . .                                                                                       | 363        |
| 16.5. Exercices . . . . .                                                                                        | 363        |
| 16.5.1. Vecteurs atomiques . . . . .                                                                             | 363        |
| 16.5.2. Listes . . . . .                                                                                         | 365        |
| 16.5.3. Tableaux de données . . . . .                                                                            | 366        |
| <b>17. Exécution conditionnelle et boucles</b>                                                                   | <b>369</b> |
| 17.1. <code>if</code> et <code>else</code> : exécuter du code sous certaines conditions . . . . .                | 369        |
| 17.1.1. <code>if</code> . . . . .                                                                                | 369        |
| 17.1.2. <code>if / else</code> . . . . .                                                                         | 371        |



|                                                                                                     |            |
|-----------------------------------------------------------------------------------------------------|------------|
| 17.1.3. “if” / “else if” / “else” . . . . .                                                         | 373        |
| 17.1.4. Construction de conditions complexes . . . . .                                              | 374        |
| 17.1.5. Différence entre if / else et ifelse . . . . .                                              | 376        |
| 17.2. Contrôle de l’exécution et gestion des erreurs . . . . .                                      | 377        |
| 17.2.1. Utilisation de <code>return</code> pour sortir de la fonction . . . . .                     | 377        |
| 17.2.2. <code>warning</code> . . . . .                                                              | 378        |
| 17.2.3. <code>stop</code> et <code>stopifnot</code> . . . . .                                       | 378        |
| 17.2.4. Tester la présence d’un argument facultatif . . . . .                                       | 380        |
| 17.3. <code>for</code> et <code>while</code> : répéter des instructions dans une boucle . . . . .   | 382        |
| 17.3.1. <code>for</code> . . . . .                                                                  | 382        |
| 17.3.2. <code>while</code> . . . . .                                                                | 385        |
| 17.3.3. <code>next</code> et <code>break</code> . . . . .                                           | 386        |
| 17.3.4. Quand (ne pas) utiliser des boucles . . . . .                                               | 387        |
| 17.4. Ressources . . . . .                                                                          | 389        |
| 17.5. Exercices . . . . .                                                                           | 390        |
| 17.5.1. <code>if</code> et <code>else</code> . . . . .                                              | 390        |
| 17.5.2. Contrôle de l’exécution . . . . .                                                           | 390        |
| 17.5.3. Boucles . . . . .                                                                           | 391        |
| <b>18. Itérer avec <code>purrr</code></b> . . . . .                                                 | <b>393</b> |
| 18.1. Exemple d’application . . . . .                                                               | 393        |
| 18.2. <code>map</code> et ses variantes . . . . .                                                   | 396        |
| 18.2.1. Modes d’appel de <code>map</code> . . . . .                                                 | 396        |
| 18.2.2. Variantes de <code>map</code> . . . . .                                                     | 398        |
| 18.2.3. <code>map</code> et tableaux de données . . . . .                                           | 400        |
| 18.3. Itérer sur les colonnes d’un tableau de données . . . . .                                     | 402        |
| 18.4. <code>modify</code> . . . . .                                                                 | 404        |
| 18.5. <code>imap</code> . . . . .                                                                   | 405        |
| 18.6. <code>walk</code> . . . . .                                                                   | 408        |
| 18.7. <code>map2</code> et <code>pmap</code> : itérer sur plusieurs vecteurs en parallèle . . . . . | 409        |
| 18.8. Répéter une opération . . . . .                                                               | 413        |
| 18.9. Quand (ne pas) utiliser <code>map</code> . . . . .                                            | 414        |
| 18.10 <code>purrr</code> vs <code>*apply</code> . . . . .                                           | 415        |
| 18.11 Ressources . . . . .                                                                          | 416        |
| 18.12 Exercices . . . . .                                                                           | 416        |
| 18.12.1 <code>map</code> et ses variantes . . . . .                                                 | 416        |
| 18.12.2 <code>modify</code> et itération sur les colonnes d’un tableau . . . . .                    | 418        |
| 18.12.3 <code>imap</code> et <code>walk</code> . . . . .                                            | 418        |
| 18.12.4 <code>map2</code> et <code>pmap</code> . . . . .                                            | 420        |
| 18.12.5 Répéter une opération . . . . .                                                             | 422        |
| <b>19. Programmer avec le <i>tidyverse</i></b> . . . . .                                            | <b>425</b> |
| 19.1. Spécificités du <i>tidyverse</i> . . . . .                                                    | 425        |
| 19.1.1. <i>data masking</i> . . . . .                                                               | 426        |
| 19.1.2. <i>tidy selection</i> . . . . .                                                             | 429        |
| 19.1.3. Utilisation dans des fonctions . . . . .                                                    | 430        |
| 19.2. Programmer avec <code>dplyr</code> et <code>tidyr</code> . . . . .                            | 431        |
| 19.2.1. Utiliser une colonne passée en argument . . . . .                                           | 431        |
| 19.2.2. Utiliser une sélection de colonnes passée en argument . . . . .                             | 432        |
| 19.2.3. Nommer de nouvelles colonnes à partir d’un argument . . . . .                               | 434        |

|                                                                             |            |
|-----------------------------------------------------------------------------|------------|
| 19.2.4. Désambiguïser data et env . . . . .                                 | 436        |
| 19.2.5. Quand les arguments sont des chaînes de caractères . . . . .        | 436        |
| 19.3. Programmer avec <code>ggplot2</code> . . . . .                        | 439        |
| 19.3.1. Sélection de colonnes avec <code>aes()</code> . . . . .             | 439        |
| 19.3.2. Faceting . . . . .                                                  | 440        |
| 19.4. Aide-mémoire . . . . .                                                | 443        |
| 19.5. Ressources . . . . .                                                  | 445        |
| 19.6. Exercices . . . . .                                                   | 446        |
| 19.6.1. <code>dplyr</code> et <code>tidyr</code> . . . . .                  | 446        |
| 19.6.2. Noms de colonnes en chaînes de caractères . . . . .                 | 448        |
| 19.6.3. Ambiguïté <code>data</code> / <code>env</code> . . . . .            | 448        |
| 19.6.4. <code>ggplot2</code> . . . . .                                      | 450        |
| <b>20. Débuggage et performance</b>                                         | <b>455</b> |
| 20.1. Débugguer une fonction . . . . .                                      | 455        |
| 20.1.1. <code>print()</code> . . . . .                                      | 455        |
| 20.1.2. Localiser une erreur . . . . .                                      | 458        |
| 20.1.3. <code>browser()</code> . . . . .                                    | 461        |
| 20.2. benchmarking : mesurer et comparer les temps d'exécution . . . . .    | 463        |
| 20.3. Quelques conseils d'optimisation . . . . .                            | 466        |
| 20.3.1. Privilégier les fonctions vectorisées . . . . .                     | 466        |
| 20.3.2. Mettre le minimum d'opérations dans les boucles . . . . .           | 467        |
| 20.3.3. Choisir un format de fichier adapté . . . . .                       | 468        |
| 20.3.4. Repérer les opérations coûteuses avec du <i>profiling</i> . . . . . | 469        |
| 20.3.5. Mettre en cache des résultats intermédiaires . . . . .              | 470        |
| 20.3.6. Utiliser d'autres extensions . . . . .                              | 471        |
| 20.4. Ressources . . . . .                                                  | 471        |
| <b>21. Organiser un projet avec targets</b>                                 | <b>473</b> |
| 21.1. Définition du pipeline . . . . .                                      | 473        |
| 21.1.1. Projet d'exemple . . . . .                                          | 473        |
| 21.1.2. <code>_targets.R</code> . . . . .                                   | 474        |
| 21.2. Exécution du pipeline . . . . .                                       | 478        |
| 21.3. Modification du pipeline . . . . .                                    | 479        |
| 21.4. RMarkdown . . . . .                                                   | 481        |
| 21.5. Gestion des données en cache . . . . .                                | 483        |
| 21.6. Avantages et limites . . . . .                                        | 484        |
| 21.6.1. Avantages . . . . .                                                 | 484        |
| 21.6.2. Interactivité et développement du pipeline . . . . .                | 484        |
| 21.7. Ressources . . . . .                                                  | 485        |
| <b>Annexes</b>                                                              | <b>487</b> |
| <b>A. Ressources</b>                                                        | <b>487</b> |
| A.1. Aide . . . . .                                                         | 487        |
| A.1.1. Aide de R et RStudio . . . . .                                       | 487        |
| A.1.2. Aide en ligne . . . . .                                              | 488        |
| A.1.3. Antisèches . . . . .                                                 | 489        |
| A.1.4. Où poser des questions . . . . .                                     | 489        |

---

|                                                  |     |
|--------------------------------------------------|-----|
| A.2. Ouvrages, blogs, MOOCs...                   | 490 |
| A.2.1. Francophones                              | 490 |
| A.2.2. Anglophones                               | 491 |
| A.3. Extensions                                  | 491 |
| A.3.1. Où trouver des extensions intéressantes ? | 491 |
| A.3.2. L'extension <b>questionr</b>              | 492 |



# À propos de ce document

Ce document est une introduction à [R](#), logiciel libre de traitement et d'analyse de données. Il se veut le plus accessible possible, y compris pour ceux qui ne sont pas particulièrement familiers avec l'informatique. Il se base à la fois sur les fonctionnalités de R “de base”, et sur une série d'extensions regroupées sous l'appellation *tidyverse*.

Ce document *n'est pas* une introduction aux méthodes statistiques d'analyse de données.

Il est basé sur R version 4.4.2 (2024-10-31).

Ce document est régulièrement corrigé et mis à jour. La version HTML de référence est disponible [en ligne](#), et il est téléchargeable aux formats [PDF](#) et [EPUB](#).

Ce document est généré avec [quarto](#), et son code source est disponible [sur GitHub](#).

## Remerciements

Un remerciement tout particulier à mes formidables collègues Sofiane Bouzid, Behnaz Khosravi et Karine Pietropaoli pour leurs nombreux et utiles retours sur la partie *Aller plus loin*.

Ce document a également bénéficié de la relecture, des suggestions et des corrections d'Éric Wiest, Diane Rodet, Mayeul Kauffmann, Jimmy Raturat, Fabienne Marquant, Julien Biaudet, Frédérique Giraud, Joël Gombin, Milan Bouchet-Valat et Joseph Larmarange.

## Licence

Ce document est mis à disposition selon les termes de la [Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 4.0 International](#).



Figure 1.: Licence Creative Commons



**partie I.**

## **Introduction**





# 1. Présentation

## 1.1. À propos de R

R est un langage orienté vers le traitement et l'analyse quantitative de données, dérivé du langage S. Il est développé depuis les années 90 par un groupe de volontaires de différents pays et par une large communauté d'utilisateurs et utilisatrices. C'est un logiciel libre, publié sous [licence GNU GPL](#).

L'utilisation de R présente plusieurs avantages :

- c'est un logiciel multiplateforme, qui fonctionne aussi bien sur des systèmes Linux, Mac OS X ou Windows.
- c'est un logiciel libre, développé par ses utilisateurs et utilisatrices, diffusable et modifiable par tout un chacun.
- c'est un logiciel gratuit.
- c'est un logiciel puissant, dont les fonctionnalités de base peuvent être étendues à l'aide d'extensions développées par la communauté. Il en existe plusieurs milliers.
- c'est un logiciel avec d'excellentes capacités graphiques.

Comme rien n'est parfait, on peut également trouver quelques inconvénients :

- le logiciel, la documentation de référence et les principales ressources sont en anglais. Il est toutefois parfaitement possible d'utiliser R sans spécialement maîtriser cette langue et il existe de plus en plus de ressources francophones.
- R n'est pas un logiciel au sens classique du terme, mais plutôt un langage de programmation. Il fonctionne à l'aide de scripts (des petits programmes) édités et exécutés au fur et à mesure de l'analyse.
- en tant que langage de programmation, R a la réputation d'être difficile d'accès, notamment pour ceux n'ayant jamais programmé auparavant.

Ce document ne demande aucun prérequis en informatique ou en programmation. Juste un peu de motivation pour l'apprentissage du langage et, si possible, des données intéressantes sur lesquelles appliquer les connaissances acquises.

L'aspect langage de programmation et la difficulté qui en découle peuvent sembler des inconvénients importants. Le fait de structurer ses analyses sous forme de scripts (suite d'instructions effectuant les différentes opérations d'une analyse) présente cependant de nombreux avantages :

- le script conserve l'ensemble des étapes d'une analyse, de l'importation des données à leur analyse en passant par les manipulations et les recodages.
- on peut à tout moment revenir en arrière et corriger ou modifier ce qui a été fait.
- il est très rapide de réexécuter une suite d'opérations complexes.
- on peut très facilement mettre à jour les résultats en cas de modification des données sources.
- le script garantit, sous certaines conditions, la reproductibilité des résultats obtenus.

## 1.2. À propos de RStudio

RStudio n'est pas à proprement parler une interface graphique pour R, il s'agit plutôt d'un *environnement de développement intégré*, qui propose des outils facilitant l'écriture de scripts et l'usage de R au quotidien. C'est une interface bien supérieure à celles fournies par défaut lorsqu'on installe R sous Windows ou sous Mac<sup>1</sup>.

Pour paraphraser [Hadrien Commenges](#), il n'y a pas d'obligation à utiliser RStudio, mais il y a une obligation à ne pas utiliser les interfaces de R par défaut.

RStudio est également un logiciel libre et gratuit. Une version payante existe, mais elle ne propose pas de fonctionnalités indispensables.

## 1.3. À propos du *tidyverse*

Le *tidyverse* est un ensemble d'extensions pour R (code développé par la communauté permettant de rajouter des fonctionnalités à R) construites autour d'une philosophie commune et conçues pour fonctionner ensemble. Elles facilitent l'utilisation de R dans les domaines les plus courants : manipulation des données, recodages, production de graphiques, etc.

La deuxième partie de ce document est entièrement basée sur les extensions du *tidyverse*, qui est présenté plus en détail Chapitre 6.

## 1.4. Structure du document

Ce document est composé de trois grandes parties :

- Une *Introduction à R*, qui présente les bases du langage R et de l'interface RStudio
- Une *Introduction au tidyverse* qui présente cet ensemble d'extensions pour la visualisation, la manipulation des données et l'export de résultats
- Une partie *Aller plus loin* qui présente comment créer ses propres fonctions et introduit des notions de programmation plus avancées

Les personnes déjà familières avec R “de base” peuvent passer directement à l'*Introduction au tidyverse*.

## 1.5. Prérequis

Le seul prérequis pour suivre ce document est d'avoir installé R et RStudio sur votre ordinateur. Il s'agit de deux logiciels libres, gratuits, téléchargeables en ligne et fonctionnant sous PC, Mac et Linux.

Pour installer R, il suffit de se rendre sur une des pages suivantes <sup>2</sup> :

- [Installer R sous Windows](#)

---

<sup>1</sup>Sous Linux R n'est fourni que comme un outil en ligne de commande.

<sup>2</sup>Sous Linux, utilisez votre gestionnaire de packages habituel.

- [Installer R sous Mac](#)

Pour installer RStudio, rendez-vous sur [la page de téléchargement du logiciel](#) et installez la version adaptée à votre système.



**partie II.**

## **Introduction à R**



## 2. Prise en main

Une fois R et RStudio installés sur votre machine, nous n'allons pas lancer R mais plutôt RStudio.

RStudio n'est pas à proprement parler une interface graphique qui permettrait d'utiliser R de manière "classique" via la souris, des menus et des boîtes de dialogue. Il s'agit plutôt de ce qu'on appelle un *Environnement de développement intégré* (IDE) qui facilite l'utilisation de R et le développement de scripts (voir Section 1.2).

### 2.1. La console

#### 2.1.1. L'invite de commandes

Au premier lancement de RStudio, l'interface est organisée en trois grandes zones.

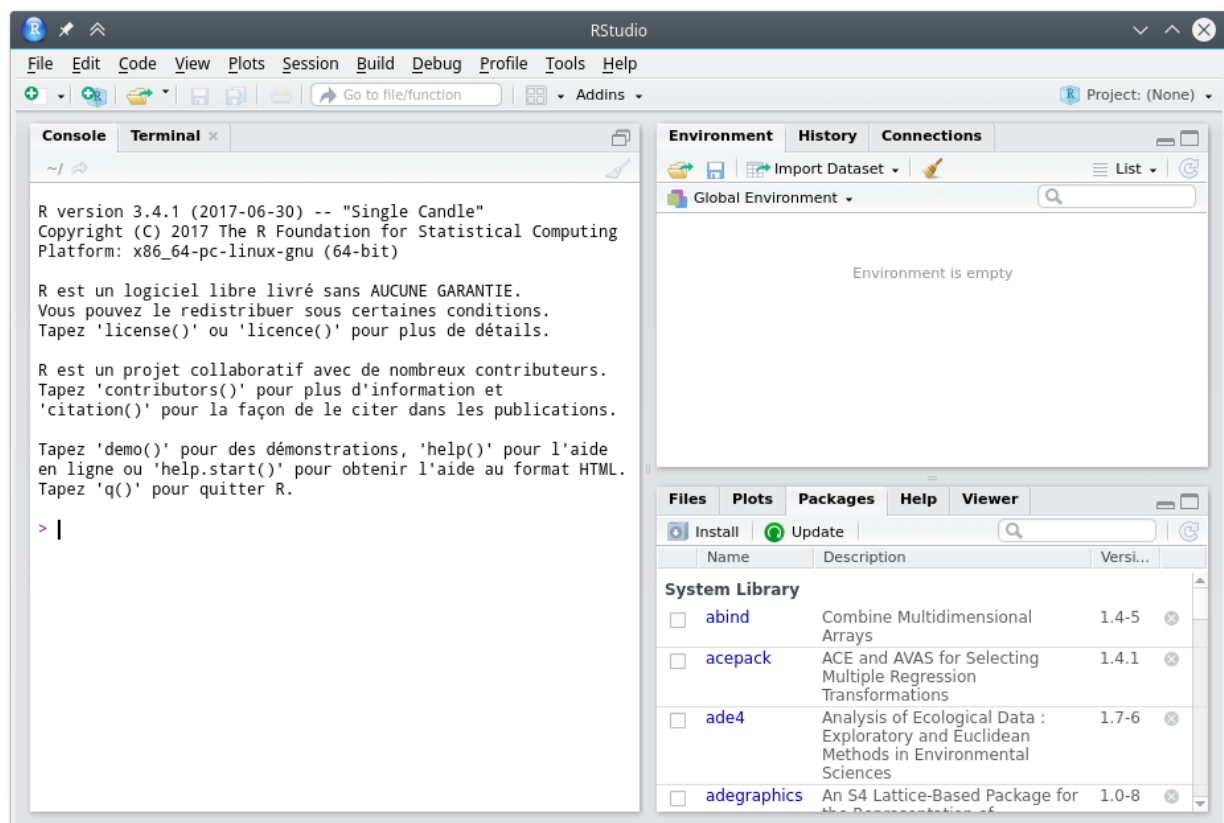


Figure 2.1.: Interface de Rstudio

La zone de gauche se nomme la *Console*. À son démarrage, RStudio a lancé une nouvelle session de R et c'est dans cette fenêtre que nous allons pouvoir interagir avec lui.

La *Console* doit normalement afficher un texte de bienvenue ressemblant à ceci :

```
R version 4.2.2 Patched (2022-11-10 r83330) -- "Innocent and Trusting"
Copyright (C) 2022 The R Foundation for Statistical Computing
Platform: x86_64-pc-linux-gnu (64-bit)
```

```
R est un logiciel libre livré sans AUCUNE GARANTIE.
Vous pouvez le redistribuer sous certaines conditions.
Tapez 'license()' ou 'licence()' pour plus de détails.
```

```
Natural language support but running in an English locale
```

```
R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.
```

```
Tapez 'demo()' pour des démonstrations, 'help()' pour l'aide
en ligne ou 'help.start()' pour obtenir l'aide au format HTML.
Tapez 'q()' pour quitter R.
```

suivi d'une ligne commençant par le caractère `>` et sur laquelle devrait se trouver votre curseur. Cette ligne est appelée l'*invite de commande* (ou *prompt* en anglais). Elle signifie que R est disponible et en attente de votre prochaine instruction.

Nous pouvons tout de suite lui fournir une première commande, en saisissant le texte suivant puis en appuyant sur **Entrée** :

```
2 + 2
#> [1] 4
```

R nous répond immédiatement, et nous pouvons constater avec soulagement qu'il sait faire des additions à un chiffre<sup>1</sup>. On peut donc continuer avec d'autres opérations :

```
5 - 7
#> [1] -2
4 * 12
#> [1] 48
-10 / 3
#> [1] -3.333333
5^2
#> [1] 25
```

Cette dernière opération utilise le symbole `^` qui représente l'opération *puissance*. `5^2` signifie donc "5 au carré".

---

<sup>1</sup>On peut ignorer pour le moment la présence du `[1]` en début de ligne.



### 2.1.2. Précisions concernant la saisie des commandes

Lorsqu'on saisit une commande, les espaces autour des opérateurs n'ont pas d'importance. Les trois commandes suivantes sont donc équivalentes, mais on privilégie en général la deuxième pour des raisons de lisibilité du code.

```
10+2
10 + 2
10      +      2
```

Quand vous êtes dans la console, vous pouvez utiliser les flèches vers le haut ↑ et vers le bas ↓ de votre clavier pour naviguer dans l'historique des commandes que vous avez tapées précédemment. Vous pouvez à tout moment modifier la commande affichée, et l'exécuter en appuyant sur **Entrée**.

Enfin, il peut arriver qu'on saisisse une commande de manière incomplète : oubli d'une parenthèse, faute de frappe, etc. Dans ce cas, R remplace l'invite de commande habituel par un signe **+**.

```
4 *
+
```

Cela signifie qu'il "attend la suite". On peut alors soit compléter la commande sur cette nouvelle ligne et appuyer sur **Entrée**, soit, si on est perdu, tout annuler et revenir à l'invite de commandes normal en appuyant sur **Esc** ou **Échap**.

## 2.2. Objets

### 2.2.1. Objets simples

Faire des calculs c'est bien, mais il serait intéressant de pouvoir stocker un résultat quelque part pour pouvoir le réutiliser ultérieurement sans avoir à faire du copier/coller.

Pour conserver le résultat d'une opération, on peut le stocker dans un *objet* à l'aide de l'opérateur d'assignation **<-**. Cette "flèche" stocke ce qu'il y a à sa droite dans un objet dont le nom est indiqué à sa gauche.

Prenons tout de suite un exemple.

```
x <- 2
```

Cette commande peut se lire *"prend la valeur 2 et mets la dans un objet qui s'appelle x"*.

Si on exécute une commande comportant juste le nom d'un objet, R affiche son contenu.

```
x
#> [1] 2
```

On voit donc que notre objet `x` contient bien la valeur 2.

On peut évidemment réutiliser cet objet dans d'autres opérations : R le remplacera alors par sa valeur.

```
x + 4
#> [1] 6
```

On peut créer autant d'objets qu'on le souhaite.

```
x <- 2
y <- 5
resultat <- x + y
resultat
#> [1] 7
```

#### Avertissement

Les noms d'objets peuvent contenir des lettres, des chiffres, les symboles `.` et `_`. Ils ne peuvent pas commencer par un chiffre. Attention, R fait la différence entre minuscules et majuscules dans les noms d'objets, ce qui signifie que `x` et `X` seront deux objets différents, tout comme `resultat` et `Resultat`.

De manière générale, il est préférable d'éviter les majuscules (pour les risques d'erreur) et les caractères accentués (pour des questions d'encodage) dans les noms d'objets.

De même, il faut essayer de trouver un équilibre entre clarté du nom (comprendre à quoi sert l'objet, ce qu'il contient) et sa longueur. Par exemple, on préférera comme nom d'objet `taille_conj1` à `taille_du_conjoint_numero_1` (trop long) ou à `t1` (pas assez explicite).

Quand on assigne une nouvelle valeur à un objet déjà existant, la valeur précédente est perdue. Les objets n'ont pas de mémoire.

```
x <- 2
x <- 5
x
#> [1] 5
```

De la même manière, assigner un objet à un autre ne crée pas de "lien" entre les deux. Cela copie juste la valeur de l'objet de droite dans celui de gauche :

```
x <- 1
y <- 3
x <- y
x
#> [1] 3
## Si on modifie y, cela ne modifie pas x
y <- 4
x
#> [1] 3
```

On le verra, les objets peuvent contenir tout un tas d'informations. Jusqu'ici on n'a stocké que des nombres, mais ils peuvent aussi contenir des chaînes de caractères (du texte), qu'on délimite avec des guillemets simples ou doubles (' ou ") :

```
chien <- "Chihuahua"
chien
#> [1] "Chihuahua"
```

### 2.2.2. Vecteurs

Imaginons maintenant qu'on a demandé la taille en centimètres de 5 personnes et qu'on souhaite calculer leur taille moyenne. On pourrait créer autant d'objets que de tailles et faire l'opération mathématique qui va bien :

```
taille1 <- 156
taille2 <- 164
taille3 <- 197
taille4 <- 147
taille5 <- 173
(taille1 + taille2 + taille3 + taille4 + taille5) / 5
#> [1] 167.4
```

Cette manière de faire n'est clairement pas pratique du tout. On va donc plutôt stocker l'ensemble de nos tailles dans un seul objet, de type *vecteur*, avec la syntaxe suivante :

```
tailles <- c(156, 164, 197, 147, 173)
```

Si on affiche le contenu de cet objet, on voit qu'il contient bien l'ensemble des tailles saisies.

```
tailles
#> [1] 156 164 197 147 173
```

Un *vecteur* dans R est un objet qui peut contenir plusieurs informations du même type, potentiellement en très grand nombre.

L'avantage d'un vecteur est que lorsqu'on lui applique une opération, celle-ci s'applique à toutes les valeurs qu'il contient. Ainsi, si on veut la taille en mètres plutôt qu'en centimètres, on peut faire :

```
tailles_m <- tailles / 100
tailles_m
#> [1] 1.56 1.64 1.97 1.47 1.73
```

Cela fonctionne pour toutes les opérations de base.

```
tailles + 10
#> [1] 166 174 207 157 183
tailles^2
#> [1] 24336 26896 38809 21609 29929
```

Imaginons maintenant qu'on a aussi demandé aux cinq mêmes personnes leur poids en kilos. On peut créer un deuxième vecteur :

```
poids <- c(45, 59, 110, 44, 88)
```

On peut alors effectuer des calculs utilisant nos deux vecteurs `tailles` et `poids`. On peut par exemple calculer l'indice de masse corporelle (IMC) de chacun de nos enquêtés en divisant leur poids en kilo par leur taille en mètre au carré :

```
imc <- poids / (tailles / 100) ^ 2
imc
#> [1] 18.49112 21.93635 28.34394 20.36189 29.40292
```

Un vecteur peut contenir des nombres, mais il peut aussi contenir du texte. Imaginons qu'on a demandé aux 5 mêmes personnes leur niveau de diplôme : on peut regrouper l'information dans un vecteur de *chaînes de caractères*. Une chaîne de caractère contient du texte libre, délimité par des guillemets simples ou doubles.

```
diplome <- c("CAP", "Bac", "Bac+2", "CAP", "Bac+3")
diplome
#> [1] "CAP"    "Bac"    "Bac+2"  "CAP"    "Bac+3"
```

L'opérateur `:`, lui, permet de générer rapidement un vecteur comprenant tous les nombres entre deux valeurs, opération assez courante sous R :

```
x <- 1:10
x
#> [1] 1 2 3 4 5 6 7 8 9 10
```

Enfin, notons qu'on peut accéder à un élément particulier d'un vecteur en faisant suivre le nom du vecteur de crochets contenant le numéro de l'élément désiré.

```
diplome[2]
#> [1] "Bac"
```

Cette opération, qui utilise l'opérateur `[]`, permet donc la sélection d'éléments d'un vecteur.

Dernière remarque, si on affiche dans la console un vecteur avec beaucoup d'éléments, ceux-ci seront répartis sur plusieurs lignes. Par exemple, si on a un vecteur de 50 nombres on peut obtenir quelque chose comme :

```
[1] 294 425 339 914 114 896 716 648 915 587 181 926 489
[14] 848 583 182 662 888 417 133 146 322 400 698 506 944
[27] 237 324 333 443 487 658 793 288 897 588 697 439 697
[40] 914 694 126 969 744 927 337 439 226 704 635
```

On remarque que R ajoute systématiquement un nombre entre crochets au début de chaque ligne : il s'agit en fait de la position du premier élément de la ligne dans le vecteur. Ainsi, le 848 de la deuxième ligne est le 14<sup>e</sup> élément du vecteur, le 914 de la dernière ligne est le 40<sup>e</sup>, etc.

Ceci explique le `[1]` qu'on obtient quand on affiche un simple nombre<sup>2</sup> :

```
[1] 4
```

---

<sup>2</sup>Et permet de constater que pour R, un nombre est un vecteur à un seul élément.

## 2.3. Fonctions

### 2.3.1. Principe

Nous savons désormais effectuer des opérations arithmétiques de base sur des nombres et des vecteurs, et stocker des valeurs dans des objets pour pouvoir les réutiliser plus tard.

Pour aller plus loin, nous devons aborder les *fonctions* qui sont, avec les objets, un deuxième concept de base de R. On utilise des fonctions pour effectuer des calculs, obtenir des résultats et accomplir des actions.

Formellement, une fonction a un *nom*, elle prend en entrée entre parenthèses un ou plusieurs *arguments* (ou *paramètres*), et retourne un *résultat*.

Prenons tout de suite un exemple. Si on veut connaître le nombre d'éléments du vecteur `tailles` que nous avons construit précédemment, on peut utiliser la fonction `length`, de cette manière :

```
length(tailles)
#> [1] 5
```

Ici, `length` est le nom de la fonction, on l'appelle en lui passant un argument entre parenthèses (en l'occurrence notre vecteur `tailles`), et elle nous renvoie un résultat, à savoir le nombre d'éléments du vecteur passé en paramètre.

Autre exemple, les fonctions `min` et `max` retournent respectivement les valeurs minimales et maximales d'un vecteur de nombres.

```
min(tailles)
#> [1] 147
max(tailles)
#> [1] 197
```

La fonction `mean` calcule et retourne la moyenne d'un vecteur de nombres.

```
mean(tailles)
#> [1] 167.4
```

La fonction `sum` retourne la somme de tous les éléments du vecteur.

```
sum(tailles)
#> [1] 837
```

Jusqu'à présent on n'a vu que des fonctions qui calculent et retournent un unique nombre. Mais une fonction peut renvoyer d'autres types de résultats. Par exemple, la fonction `range` (étendue) renvoie un vecteur de deux nombres, le minimum et le maximum.

```
range(tailles)
#> [1] 147 197
```

Ou encore, la fonction `unique`, qui supprime toutes les valeurs en double dans un vecteur, qu'il s'agisse de nombres ou de chaînes de caractères.

```
diplome <- c("CAP", "Bac", "Bac+2", "CAP", "Bac+3")
unique(diplome)
#> [1] "CAP" "Bac" "Bac+2" "Bac+3"
```

### 2.3.2. Arguments

Une fonction peut prendre plusieurs arguments, dans ce cas on les indique entre parenthèses en les séparant par des virgules.

On a déjà rencontré un exemple de fonction acceptant plusieurs arguments : la fonction `c`, qui combine l'ensemble de ses arguments en un vecteur<sup>3</sup>.

```
tailles <- c(156, 164, 197, 181, 173)
```

Ici, `c` est appelée en lui passant cinq arguments, les cinq tailles séparées par des virgules, et elle renvoie un vecteur numérique regroupant ces cinq valeurs.

Supposons maintenant que dans notre vecteur `tailles` nous avons une valeur manquante (une personne a refusé de répondre, ou notre mètre mesureur était en panne). On symbolise celle-ci dans R avec le code interne `NA`.

```
tailles <- c(156, 164, 197, NA, 173)
tailles
#> [1] 156 164 197 NA 173
```

#### Note

`NA` est l'abréviation de *Not available*, non disponible. Cette valeur particulière peut être utilisée pour indiquer une valeur manquante, qu'il s'agisse d'un nombre, d'une chaîne de caractères, etc.

Si on calcule maintenant la taille moyenne à l'aide de la fonction `mean`, on obtient :

---

<sup>3</sup>`c` est l'abréviation de *combine*, son nom est très court car on l'utilise très souvent

```
mean(tailles)
#> [1] NA
```

En effet, R considère par défaut qu'il ne peut pas calculer la moyenne si une des valeurs n'est pas disponible. Dans ce cas il considère que la moyenne est elle-même "non disponible" et renvoie donc `NA` comme résultat.

On peut cependant indiquer à `mean` d'effectuer le calcul en ignorant les valeurs manquantes. Ceci se fait en ajoutant un argument supplémentaire, nommé `na.rm` (abréviation de *NA remove*, "enlever les NA"), et de lui attribuer la valeur `TRUE` (code interne de R signifiant *vrai*).

```
mean(tailles, na.rm = TRUE)
#> [1] 172.5
```

Positionner le paramètre `na.rm` à `TRUE` indique à la fonction `mean` de ne pas tenir compte des valeurs manquantes dans le calcul.

Si on ne dit rien à la fonction `mean`, cet argument a une valeur par défaut, en l'occurrence `FALSE` (faux), qui fait qu'il ne supprime pas les valeurs manquantes. Les deux commandes suivantes sont donc rigoureusement équivalentes :

```
mean(tailles)
#> [1] NA
mean(tailles, na.rm = FALSE)
#> [1] NA
```

#### Note

Lorsqu'on passe un argument à une fonction de cette manière, c'est-à-dire sous la forme `nom = valeur`, on parle d'*argument nommé*.

### 2.3.3. Aide sur une fonction

Il est fréquent de ne pas savoir (ou d'avoir oublié) quels sont les arguments d'une fonction, ou comment ils se nomment. On peut à tout moment faire appel à l'aide intégrée à R en passant le nom de la fonction (entre guillemets) à la fonction `help`.

```
help("mean")
```

On peut aussi utiliser le raccourci `?mean`.



Ces deux commandes affichent une page (en anglais) décrivant la fonction, ses paramètres, son résultat, le tout accompagné de diverses notes, références et exemples. Ces pages d'aide contiennent à peu près tout ce que vous pourrez chercher à savoir, mais elles ne sont pas toujours d'une lecture aisée.

Dans RStudio, les pages d'aide en ligne s'ouvriront par défaut dans la zone en bas à droite, sous l'onglet *Help*. Un clic sur l'icône en forme de maison vous affichera la page d'accueil de l'aide.

## 2.4. Regrouper ses commandes dans des scripts

Jusqu'ici on a utilisé R de manière “interactive”, en saisissant des commandes directement dans la console. Ça n'est cependant pas la manière dont on va utiliser R au quotidien, pour une raison simple : lorsque R ou RStudio redémarre, tout ce qui a été effectué dans la console est perdu.

Plutôt que de saisir nos commandes dans la console, on va donc les regrouper dans des scripts (de simples fichiers texte), qui vont garder une trace de toutes les opérations effectuées, et ce sont ces scripts, sauvegardés régulièrement, qui seront le “coeur” de notre travail. C'est en ouvrant les scripts et en réexécutant les commandes qu'ils contiennent qu'on pourra “reproduire” le chargement des données, leur traitement, les analyses et leurs résultats.

Pour créer un script, il suffit de sélectionner le menu *File*, puis *New file* et *R script*. Une quatrième zone apparaît alors en haut à gauche de l'interface de RStudio. On peut enregistrer notre script à tout moment dans un fichier avec l'extension *.R*, en cliquant sur l'icône de disquette ou en choisissant *File* puis *Save*.

Un script est un fichier texte brut, qui s'édite de la manière habituelle. À la différence de la console, quand on appuie sur **Entrée**, cela n'exécute pas la commande en cours mais insère un saut de ligne (comme on pouvait s'y attendre).

Pour exécuter une commande saisie dans un script, il suffit de positionner le curseur sur la ligne de la commande en question, et de cliquer sur le bouton *Run* dans la barre d'outils juste au-dessus de la zone d'édition du script. On peut aussi utiliser le raccourci clavier **Ctrl + Entrée** (**Cmd + Entrée** sous Mac). On peut enfin sélectionner plusieurs lignes avec la souris ou le clavier et cliquer sur *Run* (ou utiliser le raccourci clavier), et l'ensemble des lignes est exécuté d'un coup.

Au final, un script pourra ressembler à quelque chose comme ça :

```
tailles <- c(156, 164, 197, 147, 173)
poids <- c(45, 59, 110, 44, 88)

mean(tailles)
mean(poids)

imc <- poids / (tailles / 100) ^ 2
min(imc)
max(imc)
```

### 2.4.1. Commentaires

Les commentaires sont un élément très important d'un script. Il s'agit de texte libre, ignoré par R, et qui permet de décrire les étapes du script, sa logique, les raisons pour lesquelles on a procédé de telle ou telle manière... Il est primordial de documenter ses scripts à l'aide de commentaires, car il est très facile de ne plus se retrouver dans un programme qu'on a produit soi-même, même après une courte interruption.

Pour ajouter un commentaire, il suffit de le faire précéder d'un ou plusieurs symboles `#`. En effet, dès que R rencontre ce caractère, il ignore tout ce qui se trouve derrière, jusqu'à la fin de la ligne.

On peut donc documenter le script précédent :

```
# Saisie des tailles et poids des enquêtés
tailles <- c(156, 164, 197, 147, 173)
poids <- c(45, 59, 110, 44, 88)

# Calcul des tailles et poids moyens
mean(tailles)
mean(poids)

# Calcul de l'IMC (poids en kilo divisé par les tailles en mètre au carré)
imc <- poids / (tailles / 100) ^ 2
# Valeurs extrêmes de l'IMC
min(imc)
max(imc)
```

## 2.5. Installer et charger des extensions (*packages*)

R étant un logiciel libre, il bénéficie d'un développement communautaire riche et dynamique. L'installation de base de R permet de faire énormément de choses, mais le langage dispose en plus d'un système d'extensions permettant d'ajouter facilement de nouvelles fonctionnalités. La plupart des extensions sont développées et maintenues par la communauté des utilisateurs et utilisatrices de R, et diffusées via un réseau de serveurs nommé [CRAN](https://cran.r-project.org/) (*Comprehensive R Archive Network*).

Pour installer une extension, si on dispose d'une connexion Internet, on peut utiliser le bouton *Install* de l'onglet *Packages* de RStudio.

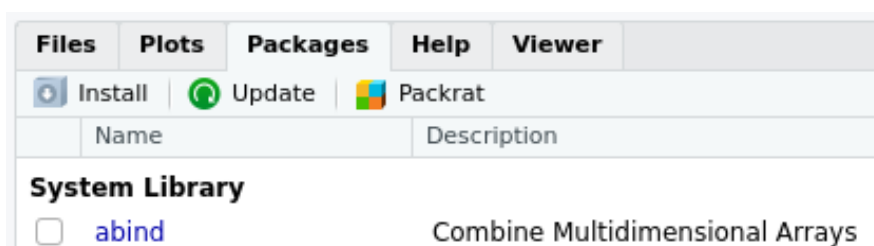


Figure 2.2.: Installer une extension

Il suffit alors d'indiquer le nom de l'extension dans le champ *Package* et de cliquer sur *Install*.

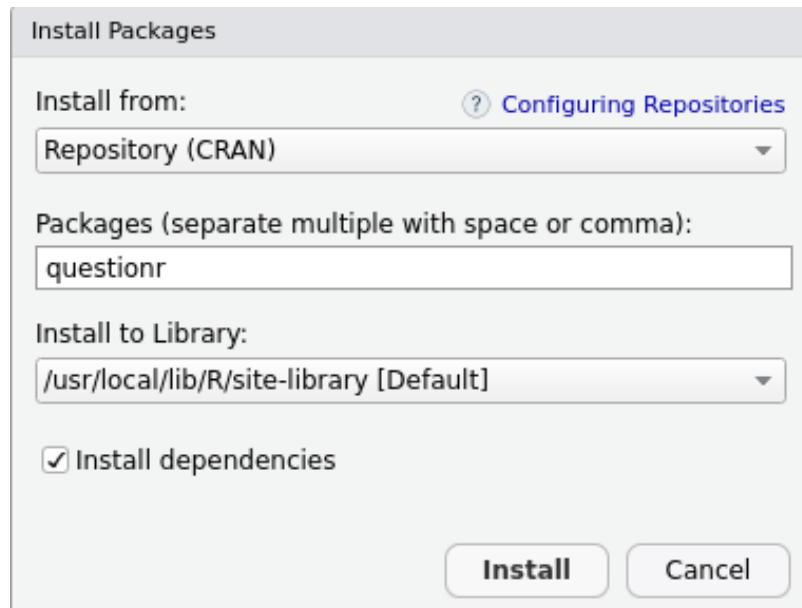


Figure 2.3.: Installation d'une extension

On peut aussi installer des extensions en utilisant la fonction `install.packages()` directement dans la console. Par exemple, pour installer le *package* `questionr` on peut exécuter la commande :

```
install.packages("questionr")
```

Installer une extension via l'une des deux méthodes précédentes va télécharger l'ensemble des fichiers nécessaires depuis l'une des machines du CRAN, puis installer tout ça sur le disque dur de votre ordinateur. Vous n'avez besoin de le faire qu'une fois, comme vous le faites pour installer un programme sur votre Mac ou PC.

Une fois l'extension installée, il faut la “charger” avant de pouvoir utiliser les fonctions qu'elle propose. Ceci se fait avec la fonction `library`. Par exemple, pour pouvoir utiliser les fonctions de `questionr`, vous devrez exécuter la commande suivante :

```
library(questionr)
```

Ainsi, on regroupe en général en début de script toute une série d'appels à `library` qui permettent de charger tous les packages utilisés dans le script. Quelque chose comme :

```
library(readxl)
library(ggplot2)
library(questionr)
```

Si vous essayez d'exécuter une fonction d'une extension et que vous obtenez le message d'erreur `impossible de trouver la fonction`, c'est certainement parce que vous n'avez pas exécuté la commande `library` correspondante.

## 2.6. Exercices

### Exercice 1

Construire le vecteur `x` suivant :

```
#> [1] 120 134 256 12
```

Utiliser ce vecteur `x` pour générer les deux vecteurs suivants :

```
#> [1] 220 234 356 112  
#> [1] 240 268 512 24
```

### Exercice 2

On a demandé à 4 ménages le revenu des deux conjoints, et le nombre de personnes du ménage :

```
conjoint1 <- c(1200, 1180, 1750, 2100)  
conjoint2 <- c(1450, 1870, 1690, 0)  
nb_personnes <- c(4, 2, 3, 2)
```

Calculer le revenu total de chaque ménage, puis diviser par le nombre de personnes pour obtenir le revenu par personne de chaque ménage.

### Exercice 3

Dans l'exercice précédent, calculer le revenu minimum et maximum parmi ceux du premier conjoint.

```
conjoint1 <- c(1200, 1180, 1750, 2100)
```

Recommencer avec les revenus suivants, parmi lesquels l'un des enquêtés n'a pas voulu répondre :

```
conjoint1 <- c(1200, 1180, 1750, NA)
```

### Exercice 4

Les deux vecteurs suivants représentent les précipitations (en mm) et la température (en °C) moyennes pour chaque mois de l'année pour la ville de Lyon (moyennes calculées sur la période 1981-2010) :

```
temperature <- c(3.4, 4.8, 8.4, 11.4, 15.8, 19.4, 22.2, 21.6, 17.6, 13.4, 7.6, 4.4)
precipitations <- c(47.2, 44.1, 50.4, 74.9, 90.8, 75.6, 63.7, 62, 87.5, 98.6, 81.9, 55.2)
```

Calculer la température moyenne sur l'année.

Calculer la quantité totale de précipitations sur l'année.

À quoi correspond et comment peut-on interpréter le résultat de la fonction suivante ? Vous pouvez vous aider de la page d'aide de la fonction si nécessaire.

```
cumsum(precipitations)
#> [1] 47.2 91.3 141.7 216.6 307.4 383.0 446.7 508.7 596.2 694.8 776.7 831.9
```

Même question pour :

```
diff(temperature)
#> [1] 1.4 3.6 3.0 4.4 3.6 2.8 -0.6 -4.0 -4.2 -5.8 -3.2
```

### Exercice 5

On a relevé les notes en maths, anglais et sport d'une classe de 6 élèves et on a stocké ces données dans trois vecteurs :

```
maths <- c(12, 16, 8, 18, 6, 10)
anglais <- c(14, 9, 13, 15, 17, 11)
sport <- c(18, 11, 14, 10, 8, 12)
```

Calculer la moyenne des élèves de la classe en anglais.

Calculer la moyenne générale de chaque élève (la moyenne des ses notes dans les trois matières).

Essayez de comprendre le résultat des deux fonctions suivantes (vous pouvez vous aider de la page d'aide de ces fonctions) :

```
pmin(maths, anglais, sport)
#> [1] 12 9 8 10 6 10
```

```
pmax(maths, anglais, sport)
#> [1] 18 16 14 18 17 12
```



## 3. Premier travail avec des données

### 3.1. Jeu de données d'exemple

Dans cette partie nous allons (enfin) travailler sur des “vraies” données, et utiliser un jeu de données présent dans l’extension `questionr`. Nous devons donc avant toute chose installer cette extension.

Pour installer ce package, deux possibilités :

- Dans l’onglet *Packages* de la zone de l’écran en bas à droite, cliquez sur le bouton *Install*. Dans le dialogue qui s’ouvre, entrez “questionr” dans le champ *Packages* puis cliquez sur *Install*.
- Saisissez directement la commande suivante dans la console : `install.packages("questionr")`

Dans les deux cas, tout un tas de messages devraient s’afficher dans la console. Attendez que l’invite de commandes `>` apparaisse à nouveau.

Pour plus d’informations sur les extensions et leur installation, voir Section 2.5.

Le jeu de données que nous allons utiliser est un extrait de l’enquête *Histoire de vie* réalisée par l’INSEE en 2003. Il contient 2000 individus et 20 variables. Pour une description plus complète et une liste des variables, voir la Section A.3.2.2.

Pour pouvoir utiliser ces données, il faut d’abord charger l’extension `questionr` (après l’avoir installée, bien entendu) :

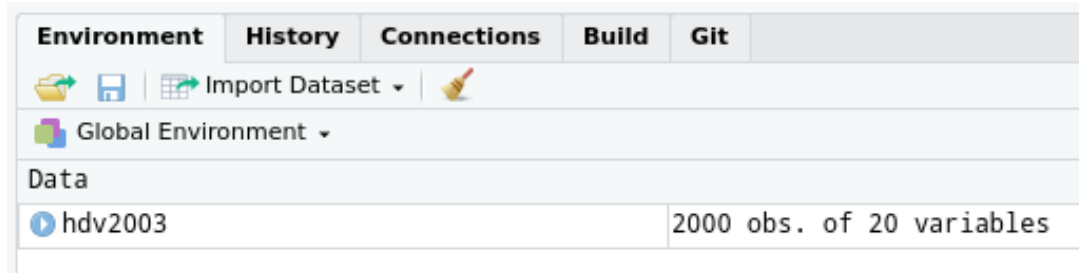
```
library(questionr)
```

L’utilisation de `library` permet de rendre “disponibles”, dans notre session R, les fonctions et jeux de données inclus dans l’extension.

Nous devons ensuite indiquer à R que nous souhaitons accéder au jeu de données à l’aide de la commande `data` :

```
data(hdv2003)
```

Cette commande ne renvoie aucun résultat particulier (sauf en cas d’erreur), mais vous devriez voir apparaître dans l’onglet *Environment* de RStudio un nouvel objet nommé `hdv2003`.

Figure 3.1.: Onglet *Environment*

Cet objet est d'un type nouveau : il s'agit d'un tableau de données.

### 3.2. Tableau de données (*data frame*)

Un *data frame* (ou tableau de données, ou table) est un type d'objet R qui contient des données au format tabulaire, avec les observations en ligne et les variables en colonnes, comme dans une feuille de tableur de type LibreOffice ou Excel.

Si on se contente d'exécuter le nom de notre tableau de données R va, comme à son habitude, nous l'afficher dans la console, ce qui est tout sauf utile.

```
hdv2003
```

Une autre manière d'afficher le contenu du tableau est de cliquer sur l'icône en forme de tableau à droite du nom de l'objet dans l'onglet *Environment* :

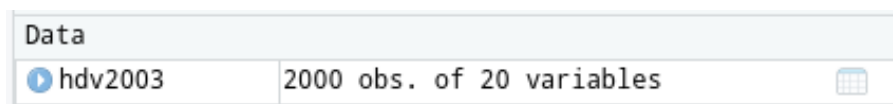


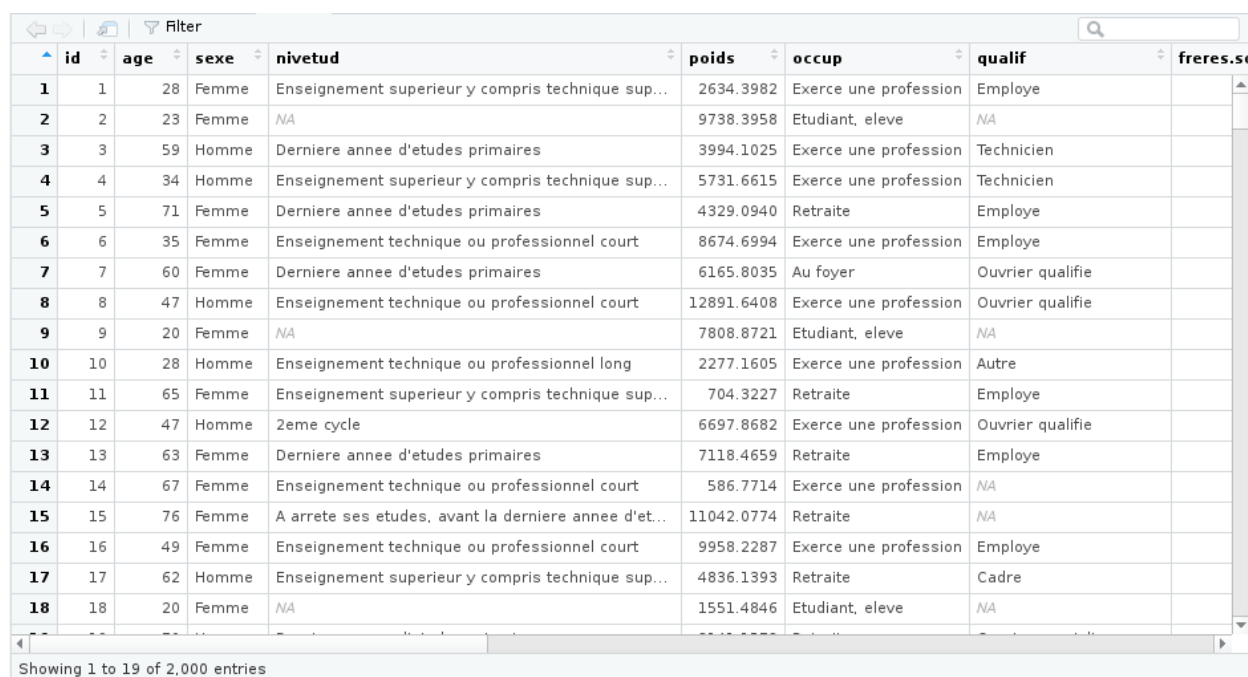
Figure 3.2.: View icon

Ou d'utiliser la fonction `View` :

```
View(hdv2003)
```

Dans les deux cas votre tableau devrait s'afficher dans RStudio avec une interface de type tableur :





|    | id | age | sexe  | nivetud                                              | poids      | occup                 | qualif           | freres.sc |
|----|----|-----|-------|------------------------------------------------------|------------|-----------------------|------------------|-----------|
| 1  | 1  | 28  | Femme | Enseignement superieur y compris technique sup...    | 2634.3982  | Exerce une profession | Employe          |           |
| 2  | 2  | 23  | Femme | NA                                                   | 9738.3958  | Etudiant, eleve       | NA               |           |
| 3  | 3  | 59  | Homme | Derniere annee d'etudes primaires                    | 3994.1025  | Exerce une profession | Technicien       |           |
| 4  | 4  | 34  | Homme | Enseignement superieur y compris technique sup...    | 5731.6615  | Exerce une profession | Technicien       |           |
| 5  | 5  | 71  | Femme | Derniere annee d'etudes primaires                    | 4329.0940  | Retraite              | Employe          |           |
| 6  | 6  | 35  | Femme | Enseignement technique ou professionnel court        | 8674.6994  | Exerce une profession | Employe          |           |
| 7  | 7  | 60  | Femme | Derniere annee d'etudes primaires                    | 6165.8035  | Au foyer              | Ouvrier qualifie |           |
| 8  | 8  | 47  | Homme | Enseignement technique ou professionnel court        | 12891.6408 | Exerce une profession | Ouvrier qualifie |           |
| 9  | 9  | 20  | Femme | NA                                                   | 7808.8721  | Etudiant, eleve       | NA               |           |
| 10 | 10 | 28  | Homme | Enseignement technique ou professionnel long         | 2277.1605  | Exerce une profession | Autre            |           |
| 11 | 11 | 65  | Femme | Enseignement superieur y compris technique sup...    | 704.3227   | Retraite              | Employe          |           |
| 12 | 12 | 47  | Homme | 2eme cycle                                           | 6697.8682  | Exerce une profession | Ouvrier qualifie |           |
| 13 | 13 | 63  | Femme | Derniere annee d'etudes primaires                    | 7118.4659  | Retraite              | Employe          |           |
| 14 | 14 | 67  | Femme | Enseignement technique ou professionnel court        | 586.7714   | Exerce une profession | NA               |           |
| 15 | 15 | 76  | Femme | A arrete ses etudes, avant la derniere annee d'et... | 11042.0774 | Retraite              | NA               |           |
| 16 | 16 | 49  | Femme | Enseignement technique ou professionnel court        | 9958.2287  | Exerce une profession | Employe          |           |
| 17 | 17 | 62  | Homme | Enseignement superieur y compris technique sup...    | 4836.1393  | Retraite              | Cadre            |           |
| 18 | 18 | 20  | Femme | NA                                                   | 1551.4846  | Etudiant, eleve       | NA               |           |

Showing 1 to 19 of 2,000 entries

Figure 3.3.: Interface “View”

Il est important de comprendre que l’objet `hdv2003` contient *l’intégralité* des données du tableau. On voit donc qu’un objet peut contenir des données de types très différents (simple nombre, texte, vecteur, tableau de données entier), et être potentiellement de très grande taille<sup>1</sup>.

#### Note

Sous R, on peut importer ou créer autant de tableaux de données qu’on le souhaite, dans les limites des capacités de sa machine.

Un *data frame* peut être manipulé comme les autres objets vus précédemment. On peut par exemple faire :

```
d <- hdv2003
```

ce qui va entraîner la copie de l’ensemble de nos données dans un nouvel objet nommé `d`. Ceci peut paraître parfaitement inutile mais a en fait l’avantage de fournir un objet avec un nom beaucoup plus court, ce qui diminuera la quantité de texte à saisir par la suite.

**Pour résumer**, comme nous avons désormais décidé de saisir nos commandes dans un script et non plus directement dans la console, les premières lignes de notre fichier de travail sur les données de l’enquête *Histoire de vie* pourraient donc ressembler à ceci :

<sup>1</sup>La seule limite pour la taille d’un objet étant la mémoire vive (RAM) de la machine sur laquelle tourne la session R.

```
## Chargement des extensions nécessaires
library(questionr)

## Jeu de données hdv2003
data(hdv2003)
d <- hdv2003
```

### 3.2.1. Structure du tableau

Un tableau étant un objet comme un autre, on peut lui appliquer des fonctions. Par exemple, `nrow` et `ncol` retournent le nombre de lignes et de colonnes du tableau.

```
nrow(d)
#> [1] 2000
```

```
ncol(d)
#> [1] 20
```

La fonction `dim` renvoie ses dimensions, donc les deux nombres précédents.

```
dim(d)
#> [1] 2000 20
```

La fonction `names` retourne les noms des colonnes du tableau, c'est-à-dire la liste de nos *variables*.

```
names(d)
#> [1] "id"           "age"          "sexe"         "nivetud"
#> [5] "poids"       "occup"        "qualif"       "freres.soeurs"
#> [9] "clso"        "relig"        "trav.imp"     "trav.satisf"
#> [13] "hard.rock"   "lecture.bd"   "peche.chasse" "cuisine"
#> [17] "bricol"      "cinema"       "sport"        "heures.tv"
```

Enfin, la fonction `str` renvoie un descriptif plus détaillé de la structure du tableau. Elle liste les différentes variables, indique leur type <sup>2</sup> et affiche les premières valeurs.

---

<sup>2</sup>Les différents types de variables seront décrits plus en détail Chapitre 9 sur les recodages.



### 3.2.2. Accéder aux variables d'un tableau

Une opération très importante est l'accès aux variables du tableau (à ses colonnes) pour pouvoir les manipuler, effectuer des calculs, etc. On utilise pour cela l'opérateur `$`, qui permet d'accéder aux colonnes du tableau. Ainsi, si l'on tape :

```
d$sexe
#> [1] Femme Femme Homme Homme Femme Femme Femme Homme Femme Homme Femme Homme
#> [13] Femme Femme Femme Femme Homme Femme Homme Femme Femme Homme Femme Femme
#> [25] Femme Homme Femme Homme Homme Homme Homme Homme Homme Homme Femme Femme
#> [37] Homme Femme Femme Homme Femme Homme Homme Femme Femme Homme Femme Femme
#> [49] Femme Femme Homme Femme Homme Femme Homme Femme Femme Femme Femme Homme Femme
#> [61] Femme Homme Homme Homme Homme Femme Homme Homme Femme Femme Femme Homme Homme
#> [73] Femme Femme Femme Femme Homme Femme Femme Femme Femme Femme Femme Femme Homme
#> [85] Homme Femme Homme Homme Homme Homme Homme Femme Homme Femme Femme Femme
#> [97] Homme Homme Femme Femme Femme Homme Femme Homme Homme Homme Femme Femme Femme
#> [109] Femme Homme Homme Homme Homme Homme Femme Homme Homme Femme Homme Homme
#> [121] Femme Femme Femme Homme Femme Femme Homme Femme Femme Homme Femme Homme
#> [133] Femme Femme Femme Homme Homme Homme Homme Homme Homme Homme Homme Femme
#> [145] Homme Homme Homme Femme Femme Femme Homme Femme Femme Femme Femme Homme
#> [157] Femme Homme Homme Homme Femme Homme Femme Homme Femme Homme Homme Femme
#> [169] Femme Femme Homme Femme Homme Femme Femme Femme Femme Homme Homme Homme
#> [181] Homme Femme Femme Homme Homme Femme Femme Femme Femme Femme Homme Homme
#> [193] Femme Homme Homme Femme Homme Femme Homme Femme
#> [ reached getOption("max.print") -- omitted 1800 entries ]
#> Levels: Homme Femme
```

R va afficher l'ensemble des valeurs de la variable `sexe` dans la console, ce qui est à nouveau fort peu utile. Mais cela nous permet de constater que `d$sexe` est un vecteur de chaînes de caractères tels qu'on en a déjà rencontré précédemment.

La fonction `table$colonne` renvoie donc la colonne nommée `colonne` du tableau `table`, c'est-à-dire un vecteur, en général de nombres ou de chaînes de caractères.

Si on souhaite afficher seulement les premières ou dernières valeurs d'une variable, on peut utiliser les fonctions `head` et `tail`.

```
head(d$age)
#> [1] 28 23 59 34 71 35
```

```
tail(d$age, 10)
#> [1] 52 42 50 41 46 45 46 24 24 66
```

Le deuxième argument numérique permet d'indiquer le nombre de valeurs à afficher.

### 3.2.3. Créer une nouvelle variable

On peut aussi utiliser l'opérateur `$` pour créer une nouvelle variable dans notre tableau : pour cela, il suffit de lui assigner une valeur.

Par exemple, la variable `heures.tv` contient le nombre d'heures passées quotidiennement devant la télé.

```
head(d$heures.tv, 10)
#> [1] 0.0 1.0 0.0 2.0 3.0 2.0 2.9 1.0 2.0 2.0
```

On peut vouloir créer une nouvelle variable dans notre tableau qui contienne la même durée convertie en minutes. On va donc créer une nouvelle variable `minutes.tv` de la manière suivante :

```
d$minutes.tv <- d$heures.tv * 60
```

On peut alors constater, soit visuellement soit dans la console, qu'une nouvelle variable (une nouvelle colonne) a bien été ajoutée au tableau.

```
head(d$minutes.tv)
#> [1] 0 60 0 120 180 120
```

## 3.3. Analyse univariée

On a donc désormais accès à un tableau de données `d`, dont les lignes sont des observations (des individus enquêtés), et les colonnes des variables (des caractéristiques de chacun de ces individus), et on sait accéder à ces variables grâce à l'opérateur `$`.

Si on souhaite analyser ces variables, les méthodes et fonctions utilisées seront différentes selon qu'il s'agit d'une variable *quantitative* (variable numérique pouvant prendre un grand nombre de valeurs : l'âge, le revenu, un pourcentage...) ou d'une variable *qualitative* (variable pouvant prendre un nombre limité de valeurs appelées modalités : le sexe, la profession, le dernier diplôme obtenu, etc.).

### 3.3.1. Analyser une variable quantitative

Une variable quantitative est une variable de type numérique (un nombre) qui peut prendre un grand nombre de valeurs. On en a plusieurs dans notre jeu de données, notamment l'âge (variable `age`) ou le nombre d'heures passées devant la télé (`heures.tv`).

### 3.3.1.1. Indicateurs de centralité

Caractériser une variable quantitative, c'est essayer de décrire la manière dont ses valeurs se répartissent, ou se distribuent.

Pour cela on peut commencer par regarder les valeurs extrêmes, avec les fonctions `min`, `max` ou `range`.

```
min(d$age)
#> [1] 18
max(d$age)
#> [1] 97
range(d$age)
#> [1] 18 97
```

On peut aussi calculer des indicateurs de *centralité* : ceux-ci indiquent autour de quel nombre se répartissent les valeurs de la variable. Il y en a plusieurs, le plus connu étant la moyenne, qu'on peut calculer avec la fonction `mean`.

```
mean(d$age)
#> [1] 48.157
```

Il existe aussi la médiane, qui est la valeur qui sépare notre population en deux : on a la moitié de nos observations en-dessous, et la moitié au-dessus. Elle se calcule avec la fonction `median`.

```
median(d$age)
#> [1] 48
```

Une différence entre les deux indicateurs est que la médiane est beaucoup moins sensible aux valeurs “extrêmes” : on dit qu'elle est plus *robuste*. Ainsi, en 2019, le salaire net *moyen* des salariés à temps plein dans le secteur privé en France était de 2424 euros, tandis que le salaire net *médian* n'était que de 1940 euros. La différence étant due à des très hauts salaires qui “tirent” la moyenne vers le haut.

### 3.3.1.2. Indicateurs de dispersion

Les indicateurs de dispersion permettent de mesurer si les valeurs sont plutôt regroupées ou au contraire plutôt dispersées.

L'indicateur le plus simple est l'étendue de la distribution, qui décrit l'écart maximal observé entre les observations :

```
max(d$age) - min(d$age)
#> [1] 79
```

Les indicateurs de dispersion les plus utilisés sont la variance ou, de manière équivalente, l'écart-type (qui est égal à la racine carrée de la variance). On obtient la première avec la fonction `var`, et le second avec `sd` (abréviation de *standard deviation*).

```
var(d$age)
#> [1] 287.0249
```

```
sd(d$age)
#> [1] 16.94181
```

Plus la variance ou l'écart-type sont élevés, plus les valeurs sont dispersées autour de la moyenne. À l'inverse, plus ils sont faibles et plus les valeurs sont regroupées.

Une autre manière de mesurer la dispersion est de calculer les quartiles :

- le premier quartile est la valeur pour laquelle on a 25% des observations en dessous et 75% au dessus
- le deuxième quartile est la valeur pour laquelle on a 50% des observations en dessous et 50% au dessus (c'est donc la médiane)
- le troisième quartile est la valeur pour laquelle on a 75% des observations en dessous et 25% au dessus

On peut les calculer avec la fonction `quantile` :

```
## Premier quartile
quantile(d$age, prob = 0.25)
#> 25%
#> 35
```

```
## Troisième quartile
quantile(d$age, prob = 0.75)
#> 75%
#> 60
```

`quantile` prend deux arguments principaux : le vecteur dont on veut calculer le quantile, et un argument `prob` qui indique quel quantile on souhaite obtenir. `prob` prend une valeur entre 0 et 1 : 0.5 est la médiane, 0.25 le premier quartile, 0.1 le premier décile, etc.

Notons enfin que la fonction `summary` permet d'obtenir d'un seul coup plusieurs indicateurs classiques :

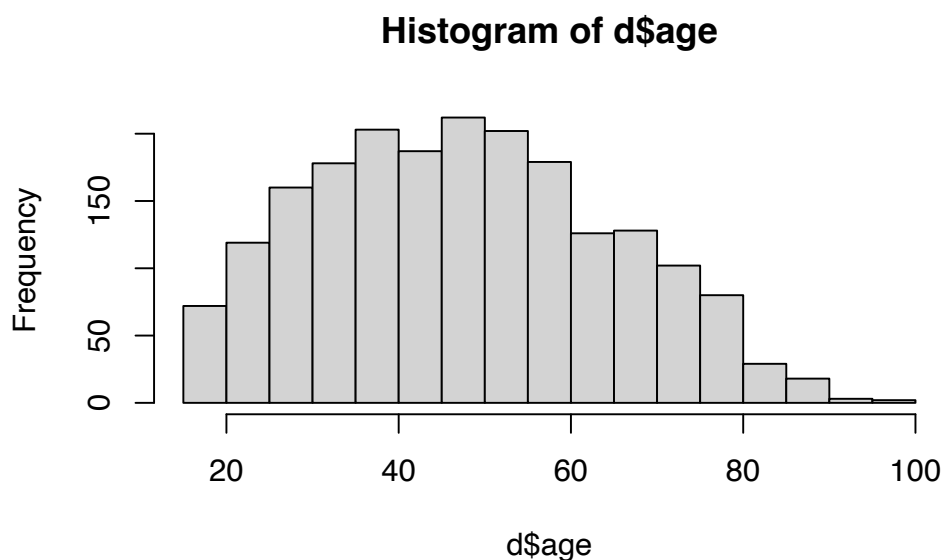
```
summary(d$age)
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>   18.00   35.00   48.00   48.16   60.00   97.00
```

### 3.3.1.3. Représentation graphique

L'outil le plus utile pour étudier la distribution des valeurs d'une variable quantitative reste la représentation graphique.

La représentation la plus courante est sans doute l'histogramme. On peut l'obtenir avec la fonction `hist`.

```
hist(d$age)
```

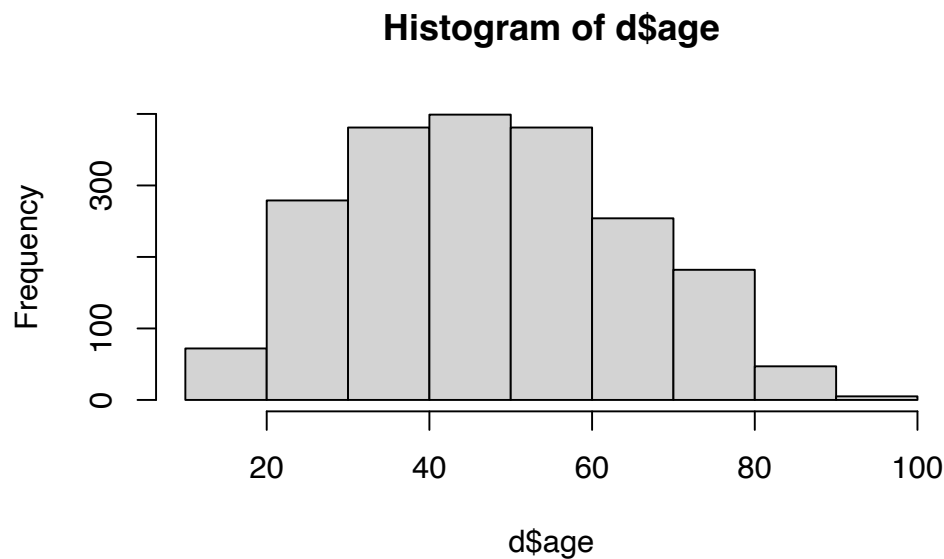


Cette fonction n'a pas pour effet direct d'effectuer un calcul ou de nous renvoyer un résultat : elle génère un graphique qui va s'afficher dans l'onglet *Plots* de RStudio.

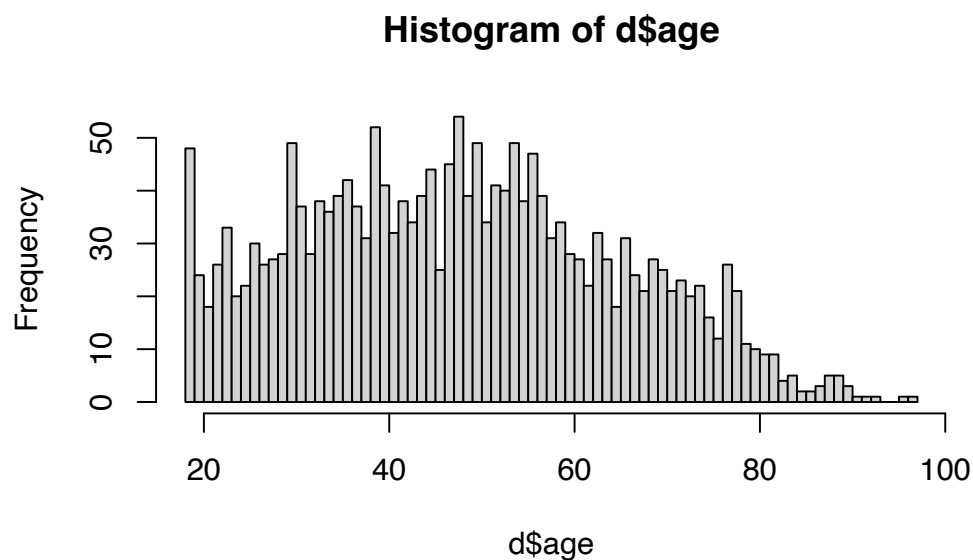
On peut personnaliser l'apparence de l'histogramme en ajoutant des arguments supplémentaires à la fonction `hist`. L'argument le plus important est `breaks`, qui permet d'indiquer le nombre de classes que l'on souhaite.

```
hist(d$age, breaks = 10)
```





```
hist(d$age, breaks = 70)
```



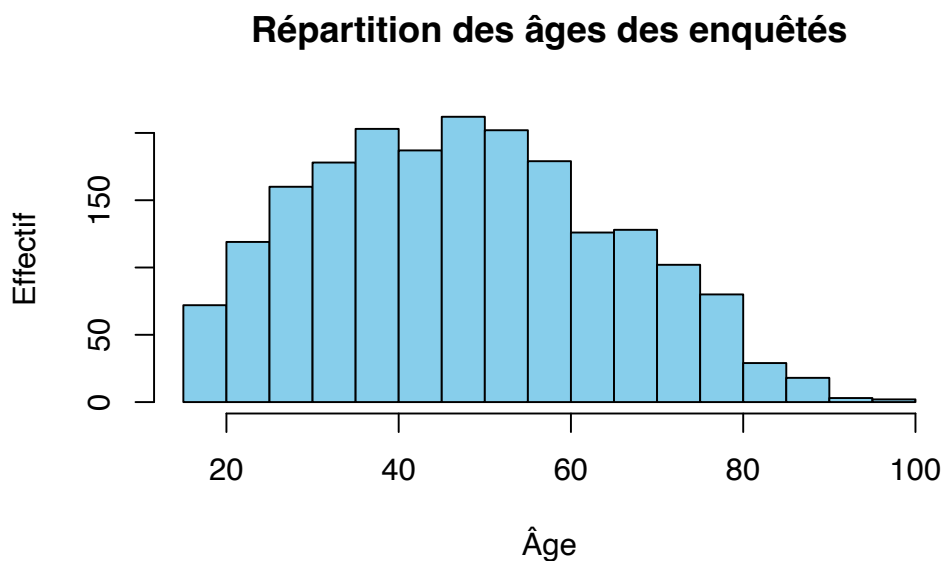
Le choix d’un “bon” nombre de classes pour un histogramme n’est pas un problème simple : si on a trop peu de classes, on risque d’effacer quasiment toutes les variations, et si on en a trop on risque d’avoir trop de détails et de masquer les grandes tendances.

Les arguments de `hist` permettent également de modifier la présentation du graphique. On peut ainsi changer la couleur des barres avec `col`<sup>3</sup>, le titre avec `main`, les étiquettes des axes avec `xlab` et `ylab`,

<sup>3</sup>Les différentes manières de spécifier des couleurs sont indiquées dans l’encadré Section 8.7.3.

etc. :

```
hist(d$age, col = "skyblue",  
     main = "Répartition des âges des enquêtés",  
     xlab = "Âge",  
     ylab = "Effectif")
```



La fonction `hist` fait partie des fonctions graphique de base de R. On verra plus en détail d'autres fonctions graphiques dans le Chapitre 8 de ce document, consacrée à l'extension `ggplot2`, qui fait partie du *tidyverse* et qui permet la production et la personnalisation de graphiques complexes.

### 3.3.2. Analyser une variable qualitative

Une variable qualitative est une variable qui ne peut prendre qu'un nombre limité de valeurs, appelées modalités. Dans notre jeu de données on trouvera par exemple le sexe (`sexe`), le niveau d'études (`nivetud`), la catégorie socio-professionnelle (`qualif`)...

À noter qu'une variable qualitative peut tout-à-fait être numérique, et que certaines variables peuvent être traitées soit comme quantitatives, soit comme qualitatives : c'est le cas par exemple du nombre d'enfants ou du nombre de frères et soeurs.

#### 3.3.2.1. Tri à plat

L'outil le plus utilisé pour représenter la répartition des valeurs d'une variable qualitative est le *tri à plat* : il s'agit simplement de compter, pour chacune des valeurs possibles de la variable (pour chacune des modalités), le nombre d'observations ayant cette valeur. Un tri à plat s'obtient sous R à l'aide de la fonction `table`.

```
table(d$sexe)
#>
#> Homme Femme
#>   899   1101
```

Ce tableau nous indique donc que parmi nos enquêtés on trouve 899 hommes et 1101 femmes.

```
table(d$qualif)
#>
#>      Ouvrier specialise      Ouvrier qualifie      Technicien
#>                203                292                86
#> Profession intermediaire      Cadre      Employe
#>                160                260                594
#>      Autre
#>                58
```

Un tableau de ce type peut être affiché ou stocké dans un objet, et on peut à son tour lui appliquer des fonctions. Par exemple, la fonction `sort` permet de trier le tableau selon la valeur de l'effectif.

```
tab <- table(d$qualif)
sort(tab)
#>
#>      Autre      Technicien Profession intermediaire
#>                58                86                160
#>      Ouvrier specialise      Cadre      Ouvrier qualifie
#>                203                260                292
#>      Employe
#>                594
```

#### Avertissement

Attention, par défaut la fonction `table` n'affiche pas les valeurs manquantes (NA). Si on souhaite les inclure il faut utiliser l'argument `useNA = "always"`, soit : `table(d$qualif, useNA = "always")`.

À noter qu'on peut aussi appliquer `summary` à une variable qualitative. Le résultat est également le tri à plat de la variable, avec en plus le nombre de valeurs manquantes éventuelles.

```
summary(d$qualif)
#>      Ouvrier specialise      Ouvrier qualifie      Technicien
#>                203                292                86
#> Profession intermediaire      Cadre      Employe
```

```
#>           160           260           594
#>      Autre      NA's
#>         58         347
```

Par défaut ces tris à plat sont en effectifs et ne sont pas toujours très lisibles, notamment quand on a des effectifs importants. On leur rajoute donc en général la répartition en pourcentages. Pour cela, nous allons utiliser la fonction `freq` de l'extension `questionr`, qui devra donc avoir précédemment été chargée avec `library(questionr)`.

```
## À rajouter en haut de script et à exécuter
library(questionr)
```

Une fois l'extension chargée on peut utiliser la fonction `freq`.

```
freq(d$qualif)
#>           n      % val%
#> Ouvrier specialise 203 10.2 12.3
#> Ouvrier qualifie 292 14.6 17.7
#> Technicien      86  4.3  5.2
#> Profession intermediaire 160  8.0  9.7
#> Cadre          260 13.0 15.7
#> Employe       594 29.7 35.9
#> Autre         58  2.9  3.5
#> NA           347 17.3  NA
```

La colonne `n` représente les effectifs de chaque catégorie, la colonne `%` le pourcentage, et la colonne `val%` le pourcentage calculé sur les valeurs valides, donc en excluant les `NA`. Une ligne a également été rajoutée pour indiquer le nombre et la proportion de `NA`.

`freq` accepte un certain nombre d'arguments pour personnaliser son affichage. Par exemple :

- `valid` indique si on souhaite ou non afficher les pourcentages sur les valeurs valides
- `cum` indique si on souhaite ou non afficher les pourcentages cumulés
- `total` permet d'ajouter une ligne avec les effectifs totaux
- `sort` permet de trier le tableau par fréquence croissante (`sort="inc"`) ou décroissante (`sort="dec"`).

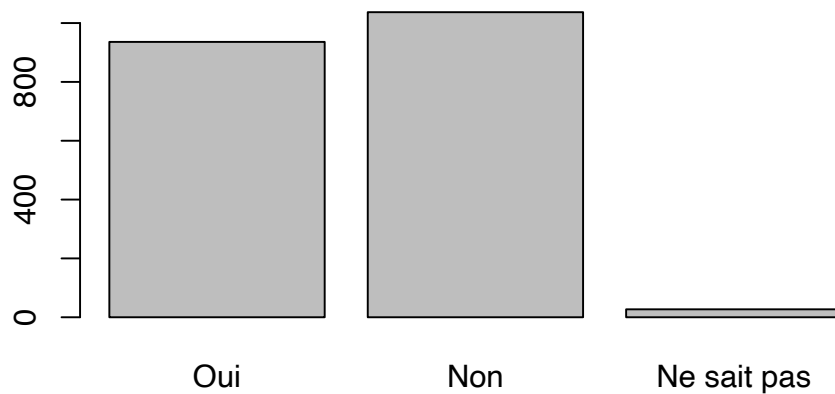
```
freq(d$qualif, valid = FALSE, total = TRUE, sort = "dec")
#>           n      %
#> Employe       594 29.7
#> Ouvrier qualifie 292 14.6
#> Cadre          260 13.0
```

```
#> Ouvrier specialise      203  10.2
#> Profession intermediaire 160   8.0
#> Technicien              86   4.3
#> Autre                   58   2.9
#> NA                     347  17.3
#> Total                   2000 100.0
```

### 3.3.2.2. Représentations graphiques

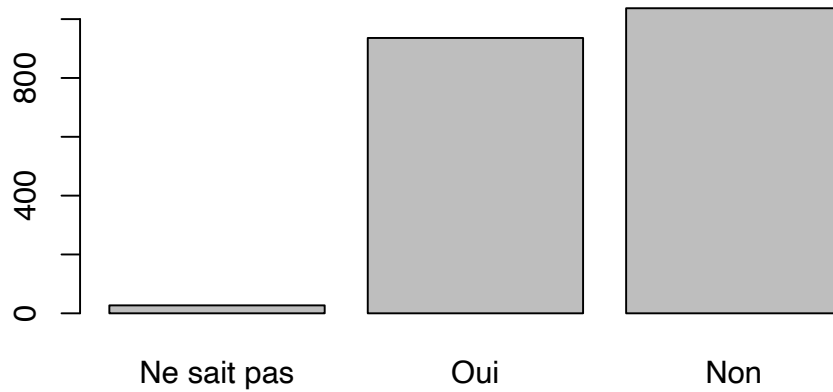
On peut représenter graphiquement le tri à plat d'une variable qualitative avec un diagramme en barres, obtenu avec la fonction `barplot`. Attention, contrairement à `hist` cette fonction ne s'applique pas directement à la variable mais au résultat du tri à plat de cette variable, calculé avec `table`. Il faut donc procéder en deux étapes.

```
tab <- table(d$clso)
barplot(tab)
```



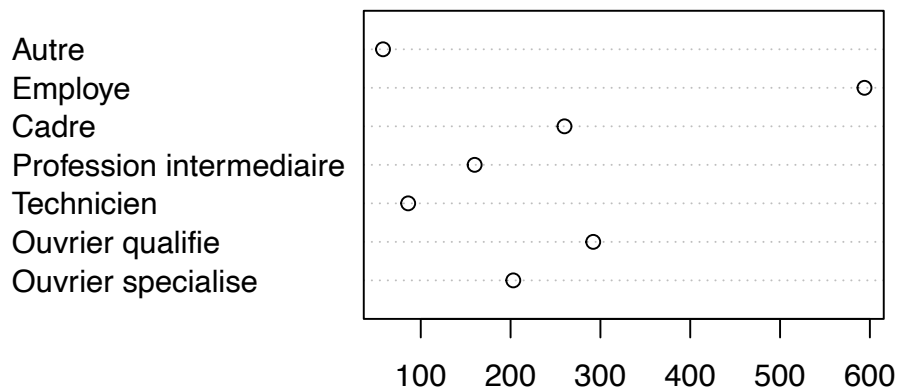
On peut aussi trier le tri à plat avec la fonction `sort` avant de le représenter graphiquement, ce qui peut faciliter la lecture du graphique :

```
barplot(sort(tab))
```



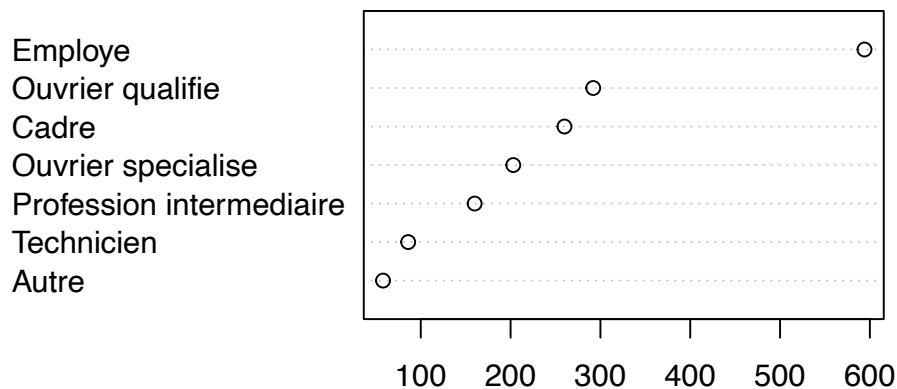
Une alternative au graphique en barres est le *diagramme de Cleveland*, qu'on peut obtenir avec la fonction `dotchart`. Celle-ci s'applique elle aussi au tri à plat de la variable calculé avec `table`.

```
dotchart(table(d$qualif))
```



Là aussi, pour améliorer la lisibilité du graphique il est préférable de trier le tri à plat de la variable avant de le représenter :

```
dotchart(sort(table(d$qualif)))
```



### 3.4. Exercices

#### Exercice 1

Créer un nouveau script qui effectue les actions suivantes :

- charger l'extension `questionr`
- charger le jeu de données nommé `hdv2003`
- copier le jeu de données dans un nouvel objet nommé `df`
- afficher les dimensions et la liste des variables de `df`

#### Exercice 2

On souhaite étudier la répartition du temps passé devant la télévision par les enquêtés (variable `heures.tv`). Pour cela, affichez les principaux indicateurs de cette variable : valeur minimale, maximale, moyenne, médiane et écart-type. Représentez ensuite sa distribution par un histogramme en 10 classes.

#### Exercice 3

On s'intéresse maintenant à l'importance accordée par les enquêtés à leur travail (variable `trav.imp`). Faites un tri à plat des effectifs des modalités de cette variable avec la commande `table`.

Faites un tri à plat affichant à la fois les effectifs et les pourcentages de chaque modalité. Y'a-t-il des valeurs manquantes ?

Représentez graphiquement les effectifs des modalités à l'aide d'un graphique en barres.

Utilisez l'argument `col` de la fonction `barplot` pour modifier la couleur du graphique en `tomato`.

Tapez `colors()` dans la console pour afficher l'ensemble des noms de couleurs disponibles dans R. Testez chaque couleur une à une pour trouver votre couleur préférée.



## 4. Analyse bivariée

Faire une analyse bivariée, c'est étudier la relation entre deux variables : sont-elles liées ? les valeurs de l'une influencent-elles les valeurs de l'autre ? ou sont-elles au contraire indépendantes ?

À noter qu'on va parler ici d'influence ou de lien, mais pas de relation de cause à effet. Les outils présentés permettent de visualiser ou de déterminer une relation, mais la mise en évidence de liens de causalité proprement dit est nettement plus complexe : il faut en effet vérifier que c'est bien telle variable qui influence telle autre et pas l'inverse, qu'il n'y a pas de "variable cachée", etc.

Là encore, le type d'analyse ou de visualisation est déterminé par la nature qualitative ou quantitative des deux variables.

### 4.1. Croisement de deux variables qualitatives

#### 4.1.1. Tableaux croisés

On continue à travailler avec le jeu de données tiré de l'enquête *Histoire de vie* inclus dans l'extension `questionr`. On commence donc par charger l'extension, le jeu de données, et à le renommer en un nom plus court pour gagner un peu de temps de saisie au clavier.

```
library(questionr)
data(hdv2003)
d <- hdv2003
```

Quand on veut croiser deux variables qualitatives, on fait un *tableau croisé*. Comme pour un tri à plat ceci s'obtient avec la fonction `table` de R, mais à laquelle on passe cette fois deux variables en argument. Par exemple, si on veut croiser la catégorie socio-professionnelle et le sexe des enquêtés :

```
table(d$qualif, d$sexe)
#>
#>               Homme Femme
#> Ouvrier specialise      96   107
#> Ouvrier qualifie     229    63
#> Technicien             66    20
#> Profession intermediaire  88    72
#> Cadre                 145   115
#> Employe              96   498
#> Autre                 21    37
```

Pour pouvoir interpréter ce tableau on doit passer du tableau en effectifs au tableau en pourcentages ligne ou colonne. Pour cela, on peut utiliser les fonctions `lprop` et `cprop` de l'extension `questionr`, qu'on applique au tableau croisé précédent.

Pour calculer les pourcentages ligne :

```
tab <- table(d$qualif, d$sexe)
lprop(tab)
#>
#>
#>      Homme Femme Total
#> Ouvrier specialise  47.3  52.7 100.0
#> Ouvrier qualifie   78.4  21.6 100.0
#> Technicien         76.7  23.3 100.0
#> Profession intermediaire 55.0  45.0 100.0
#> Cadre              55.8  44.2 100.0
#> Employe            16.2  83.8 100.0
#> Autre              36.2  63.8 100.0
#> All                44.8  55.2 100.0
```

Et pour les pourcentages colonne :

```
cprop(tab)
#>
#>
#>      Homme Femme All
#> Ouvrier specialise  13.0  11.7 12.3
#> Ouvrier qualifie   30.9   6.9 17.7
#> Technicien         8.9   2.2  5.2
#> Profession intermediaire 11.9   7.9  9.7
#> Cadre              19.6  12.6 15.7
#> Employe            13.0  54.6 35.9
#> Autre              2.8   4.1  3.5
#> Total             100.0 100.0 100.0
```

### **i** Note

Pour savoir si on doit faire des pourcentages ligne ou colonne, on pourra se référer à l'article suivant :

<https://alain-leger.lescigales.org/textes/lignecolonne.pdf>

En résumé, quand on fait un tableau croisé, celui-ci est parfaitement symétrique : on peut inverser les lignes et les colonnes, ça ne change pas son interprétation. Par contre, on a toujours en tête un “sens” de lecture dans le sens où on considère que l'une des variables *dépend* de l'autre. Par exemple, si on croise sexe et type de profession, on dira que le type de profession dépend du sexe, et non l'inverse : le type de profession est alors la variable *dépendante* (à expliquer), et le sexe la variable *indépendante* (explicative).

Pour faciliter la lecture d'un tableau croisé, il est recommandé de **faire les pourcentages sur la variable indépendante**. Dans notre exemple, la variable indépendante est le sexe, elle est

en colonne, on calcule donc les pourcentages colonnes qui permettent de comparer directement, pour chaque sexe, la répartition des catégories socio-professionnelles.

#### 4.1.2. Test du $\chi^2$

Comme on travaille sur un échantillon et pas sur une population entière, on peut compléter ce tableau croisé par un test d'indépendance du  $\chi^2$ . Celui-ci permet de tester, et éventuellement de rejeter, l'hypothèse d'indépendance des lignes et des colonnes du tableau, c'est à dire l'hypothèse que les écarts à l'indépendance observés seraient uniquement dus au biais d'échantillonnage (au fait qu'on n'a pas interrogé toute notre population).

Pour effectuer un test de ce type, on applique la fonction `chisq.test` au tableau croisé calculé précédemment.

```
chisq.test(tab)
#>
#> Pearson's Chi-squared test
#>
#> data:  tab
#> X-squared = 387.56, df = 6, p-value < 0.00000000000000022
```

Le résultat nous indique trois valeurs :

- **X-squared**, la valeur de la statistique du  $\chi^2$  pour notre tableau, c'est-à-dire une “distance” entre notre tableau observé et celui attendu si les deux variables étaient indépendantes.
- **df**, le nombre de degrés de libertés du test, qui dépend des dimensions du tableau.
- **p-value**, le fameux  $p$ , qui indique la probabilité d'obtenir une valeur de la statistique du  $\chi^2$  au moins aussi extrême sous l'hypothèse d'indépendance.

Ici, le  $p$  est extrêmement petit (la notation  $< 2.2\text{e-}16$  indique qu'il est plus petit que la plus petite valeur proche de zéro calculable par R), donc certainement en-dessous du seuil de décision choisi préalablement au test (souvent 5%, soit 0.05). On peut donc rejeter l'hypothèse d'indépendance des lignes et des colonnes du tableau.

En complément du test du  $\chi^2$ , on peut aussi regarder les *résidus* de ce test pour affiner la lecture du tableau. Ceux-ci s'obtiennent avec la fonction `chisq.residuals` de `questionr` :

```
chisq.residuals(tab)
#>
#>
#>      Homme  Femme
#> Ouvrier specialise    0.52 -0.47
#> Ouvrier qualifie    8.57 -7.73
#> Technicien    4.42 -3.98
#> Profession intermediaire  1.92 -1.73
#> Cadre    2.64 -2.38
```

```
#>   Employe      -10.43    9.41
#>   Autre       -0.98    0.88
```

L'interprétation des résidus est la suivante :

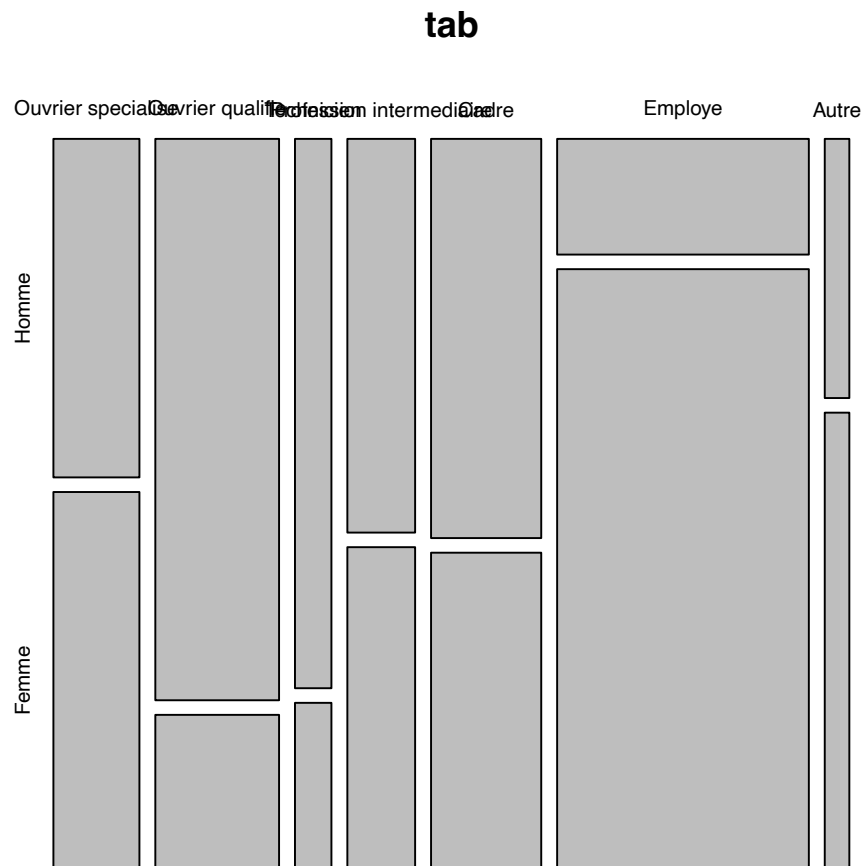
- si la valeur du résidu pour une case est inférieure à -2, alors il y a une sous-représentation de cette case dans le tableau : les effectifs sont significativement plus faibles que ceux attendus sous l'hypothèse d'indépendance
- à l'inverse, si le résidu est supérieur à 2, il y a sur-représentation de cette case
- si le résidu est compris entre -2 et 2, il n'y a pas d'écart à l'indépendance significatif

Les résidus peuvent être une aide utile à l'interprétation, notamment pour des tableaux de grande dimension.

#### 4.1.3. Représentation graphique

Il est possible de faire une représentation graphique d'un tableau croisé, par exemple avec la fonction `mosaicplot` :

```
mosaicplot(tab)
```



On peut améliorer ce graphique en colorant les cases selon les résidus du test du  $\chi^2$  (argument `shade = TRUE`) et en orientant verticalement les labels de colonnes (argument `las = 3`) :

```
mosaicplot(tab, las = 3, shade = TRUE)
```